

5

Python Programming Fundamentals

5.1 INTRODUCTION

In the previous chapter, we have discussed the two modes of interacting with Python, viz. Interactive mode and Script mode. A Python program, sometimes called a script, is a sequence of definitions and commands. These definitions are evaluated and the commands are executed by the Python interpreter, which is known as Python Shell.

Before moving further, we will discuss Python syntax. Python syntax refers to a set of rules that defines how users and the system should write and interpret a Python program. If you want to write and run your program in Python, you must familiarize yourself with its syntax. The syntax to be discussed constitutes all the basic elements of a Python program along with the concept of variables and its data types that you should understand first.

In the Python programming language, data types are inbuilt and, unlike in C++, declaration of variables is not required and memory management is automatically done by Python.

Whatever values we assign to the variables, the data type of the variable will be the same as the data type of the value. Hence, it is said that Python supports dynamic typing.

CTM: Data types are inbuilt in Python and, hence, it supports dynamic typing.

Before talking in detail about variables in Python, we will first discuss basic elements of Python, viz. character set, tokens, expressions, statements, operators and input-output, etc.

5.2 PYTHON CHARACTER SET

Character set is a set of valid characters recognized by Python. A character represents any letter, digit or any other symbol. Python uses the traditional ASCII character set. However, the latest version recognizes the Unicode character set. The ASCII character set is a subset of the Unicode character set. Python supports the following character set:

- ☛ **Letters:** A-Z, a-z
- ☛ **Digits:** 0-9
- ☛ **Special Symbols:** space + - / * \ ** () [] { } // = != == < > . " ' , ; : % ! # ? \$ & ^ <= >= @ _
- ☛ **Whitespaces:** Blank space, tabs ('\t'), carriage return (\r), newline, formfeed
- ☛ **Other Characters:** All other 256 ASCII and Unicode characters

5.3 TOKENS

A **token** is the smallest element of a **Python script** that is meaningful to the interpreter.

The following categories of tokens exist: identifiers, keywords, literals, operators and delimiters.

Identifiers: The name of any variable, constant, function, or module is called an identifier. Some of the rules for defining identifiers are:

- First character of an identifier should start with a character or an underscore (`_`) but not with a digit.
- No special characters are allowed except underscore (`_`).
- Reserve words cannot be used as an identifier name.
- Space is not allowed.
- Upper and lower case are treated differently.

Examples of valid identifiers are: `abc`, `x12y`, `india_123`, `swap`, `name`, etc.

Examples of invalid identifiers are: `12digit`, `Roll No.`, `Name$`, `Pin code`, etc.

Keywords: The reserved words of Python, which have a special fixed meaning for the interpreter, are called keywords. No keyword can be used as an identifier. The Python keywords have been listed in the successive Section 5.6.

Literals: A fixed numeric or non-numeric value is called a literal. It can be defined as a number, text, or other data that represents values to be stored in variables. They are also known as constants. Python supports the following types of literals:

- String Literals – for example, `a = "abc"`, `b = "Independence"`, etc.
- Numeric Literals – for example, `p = 2`, `a = 1000`, etc.
- Floating Literals – for example, `salary = 15000.00`, `area = 1.2`, etc.
- Boolean Literals – for example, `value = True`, `value2 = False`, etc.
- Collection literals – for example, `list1 = [1,2,3,4,5]`, `tuple1 = (10,20,30,40,50)`, etc.

Therefore, literals are data items that have fixed value.

Operators: A symbol or a word that performs some kind of operation on given values and returns the result. Examples of operators are: `+`, `-`, `**`, `/`, etc.

Delimiters: Delimiters are the symbols which can be used as separators of values or to enclose some values. Examples of delimiters are: `()` `{}` `[]` `,` `;` `:`

Note: `#` symbol used to insert comments is not a token. Any comment itself is not a token.



Fig. 5.1: Tokens in Python

For example,

Observe the following script and enlist all the tokens used in it. For each token, write its category too.

```
#Identify tokens in the script
x=68
y=12
z=x/y
print("x, y, z are:",x,y,z)
```

Token	Category
x	Identifier/Variable
y	Variable
z	Variable
print	Keyword
=	Delimiter/Operator
/	Operator
68	Literal
12	Literal
"x, y, z are:"	Literal

We will discuss these tokens in detail in the successive sub-topics. But before that, we will look into the concept of variables and data types in Python.

5.4 VARIABLES AND DATA TYPES

A variable is like a container that stores values that you can access or change. Variable, as the name suggests, has been derived from the word "vary" which means that a variable may vary during the execution of a program, i.e., the value of a variable keeps changing during the program execution.

In Python, every element is termed as an object. Hence, a variable is also an object in Python.

I am a Python variable.
My name is x and I can point
to an arbitrary object. In this
case to an int object.

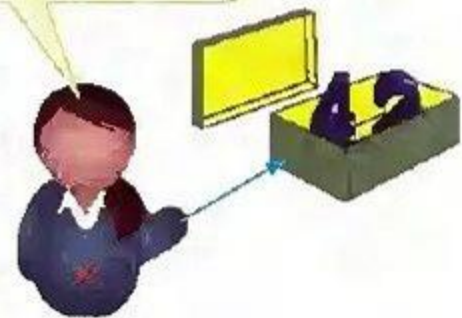


Fig. 5.2: A Variable in Python

CTM: Variables provide a way to associate names with objects.

Through a variable we store the value in the memory of a computer. Variable is a named unit of storage. A program manipulates its value. Every variable has a type and this type defines the format and behaviour of the variable.

For example, if we write

`x = 3` (as shown in the state diagram, Fig. 5.3)

it will automatically assign the value 3 to the variable named x, which is an integer, and whenever a variable name occurs in an expression, the interpreter replaces it with the value of that particular variable.

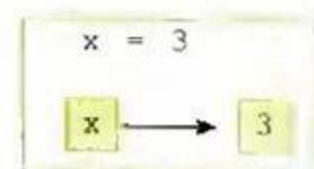
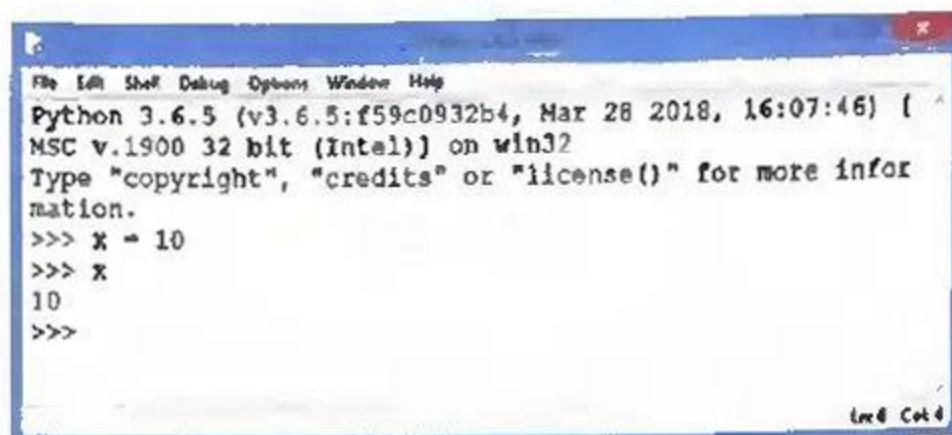


Fig. 5.3: State Diagram of Variable x

Similarly, if we type `x = 10 (>>> x=10)` and then write `x (>>> x)` and press the Enter key, it will display 10 as the output in the Python shell (Fig. 5.4).



```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [
MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more infor
mation.
>>> x = 10
>>> x
10
>>>
```

Fig. 5.4: Value of Variable x Gets Displayed

The above behaviour exhibits that variable `x` has been given a value 10, which is stored inside a memory location with the name `x`. This statement shall store 10 in `x` but does not get displayed until and unless we type `x (>>> x)`.

On typing `x`, value 10 shall be displayed as the output.

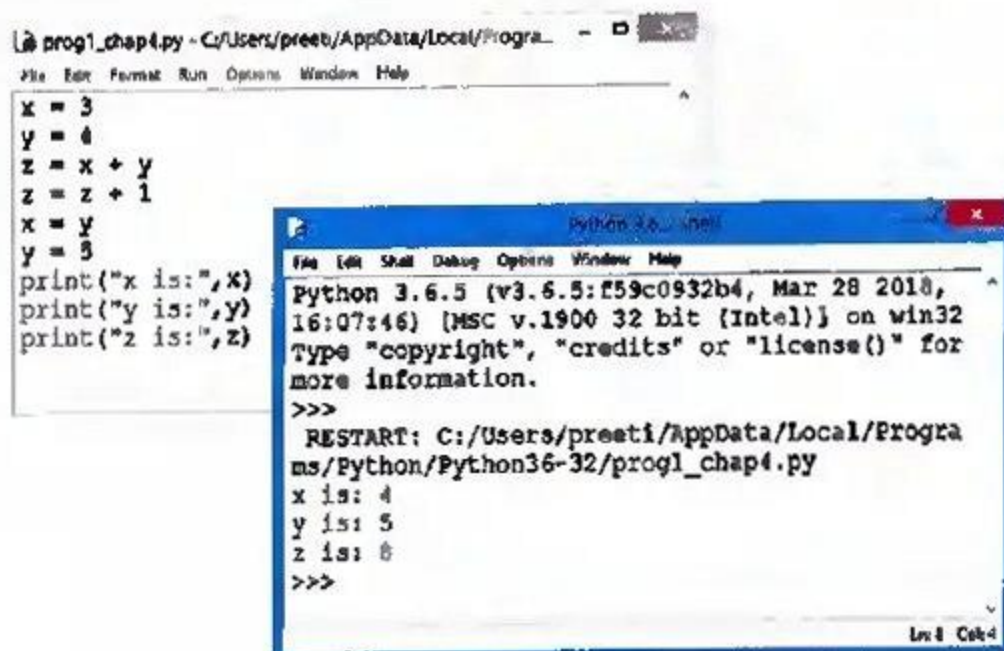
Let's take another example of working with variables. Consider the following code:

```
>>> x = 3
>>> y = 4
>>> z = x + y
>>> z = z + 1
>>> x = y
>>> y = 5
```

After executing the above lines of the code, `x` is 4, `y` is 5 and `z` is 8.

Alternatively, we can type this program in script mode also, but for displaying the values of variables `x`, `y` and `z`, we need to give `print()` statement explicitly. `print()` statement is used to display the value assigned to a variable. The `print()` statement can print multiple values on a single line. The comma (,) separates the list of values and variables that are printed.

Now save the code with a suitable file name with the extension `.py` and then run it (Run -> Run Module or press F5).



```
prog1_chap4.py - C:/Users/preeti/AppData/Local/Progra...
File Edit Format Run Options Window Help
x = 3
y = 4
z = x + y
z = z + 1
x = y
y = 5
print("x is:", x)
print("y is:", y)
print("z is:", z)

Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/prog1_chap4.py
x is: 4
y is: 5
z is: 8
>>>
```

Fig. 5.5: Running Code Inside Script Mode

Let us understand this code line by line:

1. x starts with the value 3 and y starts with the value 4.
2. In line 3, a variable z is created and equals $x + y$, which is 7.
3. Then the value of z is changed to one more than it currently equals, changing it from 7 to 8.
4. Next, x is changed to the current value of y, which is 4.
5. Finally, y is changed to 5. Note that this does not affect x.
6. So in the end, x is 4, y is 5 and z is 8.

CTM: print() statement is used to display the value assigned to a variable. The print() statement can print multiple values on a single line. The comma (,) separates the list of values and variables that are printed.

A variable/object has three main components:

- A) Identity of the Variable/Object
- B) Type of the Variable/Object
- C) Value of the Variable/Object

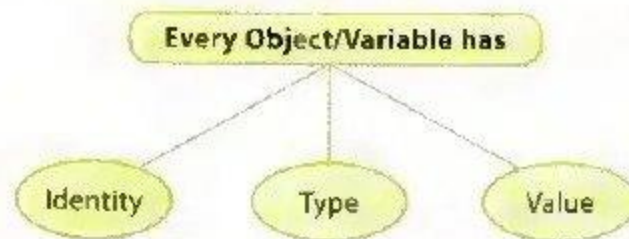


Fig. 5.6: Components of a Variable

We shall now discuss each of these associated components one by one.

A) Identity of the Variable/Object:

It refers to the object's (variable in this case) address in the memory which does not change once it has been created. The address of an object can be checked using the method `id(object)`.

Syntax: `>>> id(variable/object)`

For example, `>>> x=10`

`>>> id(x)`

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x = 10
>>> id(x)
1614996384
>>>
```

Fig. 5.7: Address of a Variable Using `id()` Method

B) Type of the Variable/Object:

By **type** we refer to the **data type** of a variable. In Python, each value is an object and has a data type associated with it. Data type of a value refers to the value it has and the allowable operations on those values. A data type in Python can be categorized as follows:

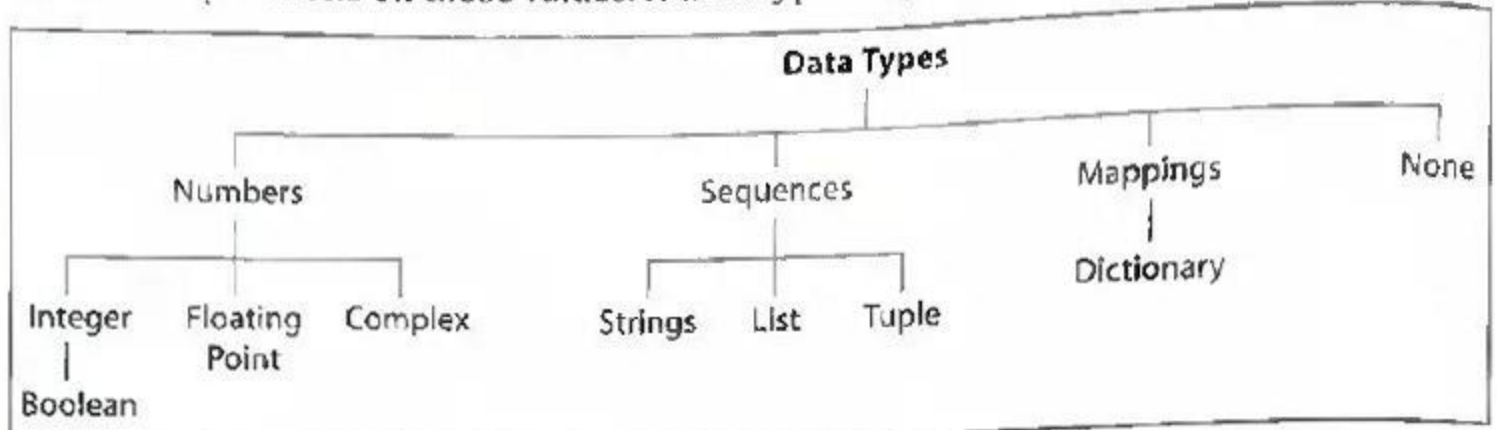


Fig. 5.8: Data Types in Python

The above figure shows the data types available in Python. We are going to discuss only fundamental data types available in Python which are as follows:

1. **Number:** Number data type stores numerical values. One of the most distinguished features of Python is that you don't really have to declare a numeric value to define its type. Python can easily differentiate one data type from another when you write and run your statement. Python supports three types of built-in numeric data types: integer/long, floating point numbers and complex numbers.

CTM: Long integers do not form a separate group of instructions but are included in integer or 'int'.

- (a) **int (integer):** Integer represents whole numbers without any fractional part. They can be positive or negative and have unlimited size in Python. Thus, while working with integers, we need not worry about the size of the integer as a very big-sized integer is automatically handled by Python.

Learning Tip: An integer in Python can be of any length.

```
Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:34:40) [MSC v.1927 64 ^
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> x=6
>>> x
6
>>> y=-255
>>> y
-255
>>> z=0
>>> z
0
>>> a=2345678
>>> a
2345678
>>> b=-67854325678
>>> b
-67854325678
>>>
```

Ln: 18 Col: 4

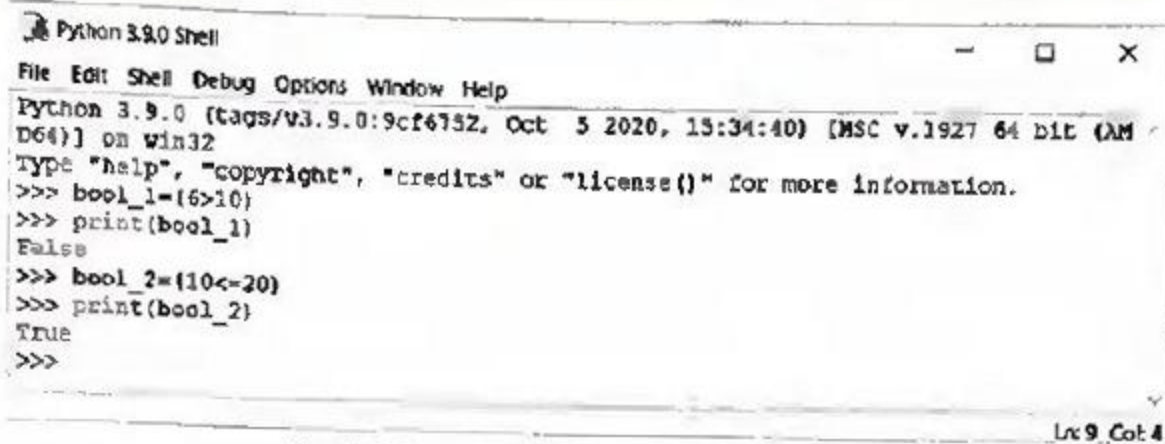
Examples of integers recognized by Python are: -6, 793, -255, 4, 0, 23466, -45766782964, etc.

While writing a large integer value, do not use commas to separate digits. Also, integers should not have leading zeroes.

CTM: Range of an integer in Python can be from -2147483648 to 2147483647, and long integer has unlimited range, subject to available memory.

- **bool (Boolean):** Boolean data type is a sub-type of integer. It represents one of the two possible values—True or False. Any expression which can be True or False has the data type bool.

CTM: A Boolean True value is non-zero, non-null and non-empty.



```
Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:34:40) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> bool_1=(6>10)
>>> print(bool_1)
False
>>> bool_2=(10<=20)
>>> print(bool_2)
True
>>>
```

Ln: 9, Col: 4

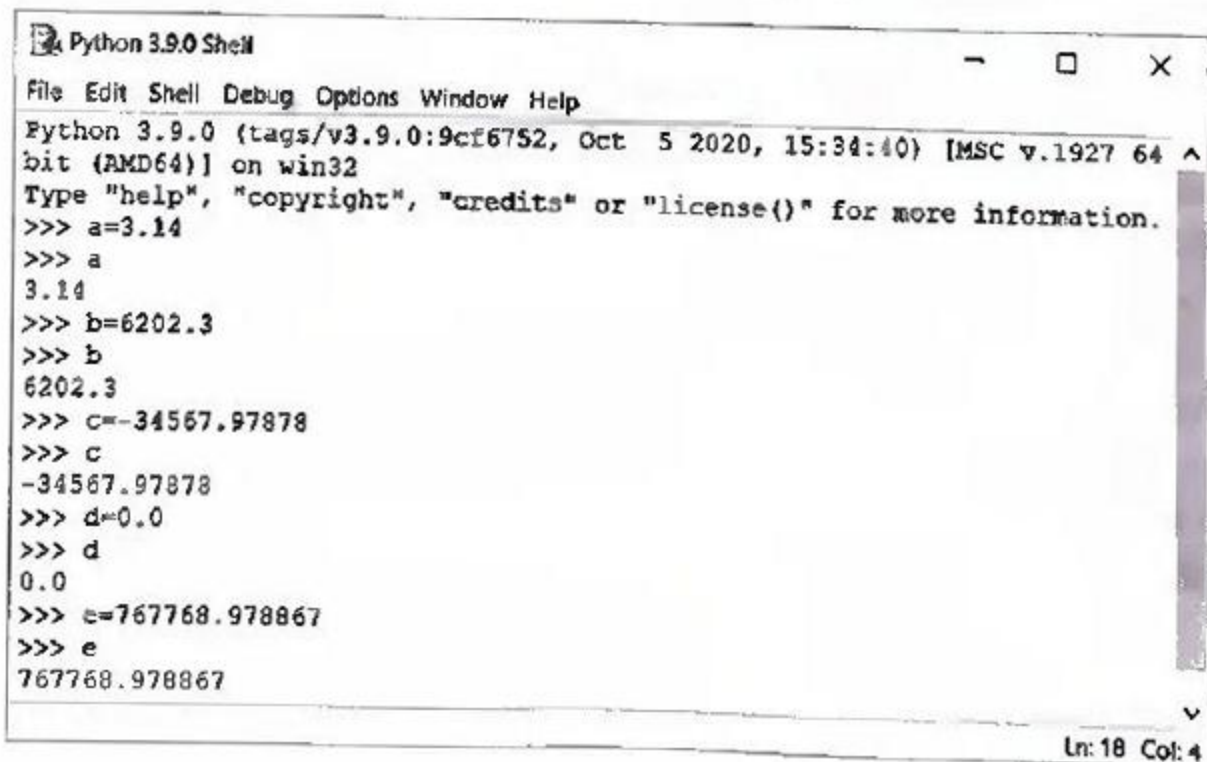
Fig. 5.9: Handling bool (Boolean) Data Type

Examples of Boolean expressions are:

- ```
>>> bool_1 = (6>10)
>>> print(bool_1)
False
```
- ```
>>> bool_2 = (10<=20)
>>> print(bool_2)
True
```

- (b) **float (floating point numbers):** Floating point numbers denote real numbers or floating point values (*i.e.*, numbers with fractional part). Floating point numbers are written with a decimal point that separates the integer from the fractional numbers.

Learning Tip: The fractional part of a floating point number can also be 0 (zero).



```
Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:34:40) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> a=3.14
>>> a
3.14
>>> b=6202.3
>>> b
6202.3
>>> c=-34567.97878
>>> c
-34567.97878
>>> d=0.0
>>> d
0.0
>>> e=767768.978867
>>> e
767768.978867
```

Ln: 18, Col: 4

Examples of floating point numbers are: 3.14, 6202.3, -43.2, 6.0, 28879.26, etc.

CTM: A value stored as a floating point number in Python can have a maximum of 53 bits of precision.

- (c) **Complex numbers:** Complex numbers in Python are made up of pairs of real and imaginary numbers. They take the form 'x + yj' or 'x + yj', where 'x' is a float and the real part of the complex number. On the other side is yj, where 'y' is a float and j, or its lowercase indicates the square root of an imaginary number -1. This makes 'y' the imaginary part of the complex number.

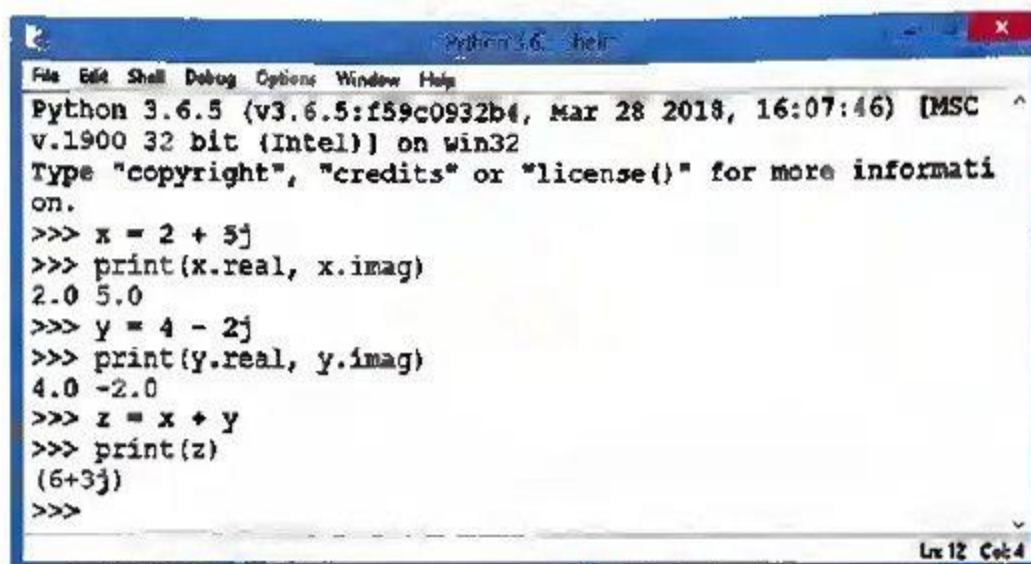
Here are some examples of complex numbers:

- (i)

```
>>> x = 2 + 5j
>>> print(x.real, x.imag)
2.0 5.0
```
- (ii)

```
>>> y = 4 - 2j
>>> print(y.real, y.imag)
4.0 -2.0
```
- (iii)

```
>>> z = x + y
>>> print(z)
(6 + 3j)
```



```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC
v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more informati
on.
>>> x = 2 + 5j
>>> print(x.real, x.imag)
2.0 5.0
>>> y = 4 - 2j
>>> print(y.real, y.imag)
4.0 -2.0
>>> z = x + y
>>> print(z)
(6+3j)
>>>
```

Fig. 5.10: Handling Complex Numbers

Complex numbers are not extensively used in Python programming.

2. **Sequence:** A sequence is a group of items with a well-defined ordering where each item is indexed by an integer. The three sequence data types available in Python are Strings, Lists and Tuples. We will learn about them in detail in later chapters. A brief introduction to these data types is as follows:

- (a) **str (String):** A string is a sequence of characters that can be a combination of letters, numbers and special symbols, enclosed within quotation marks, single or double or triple (' ' or " " or "" "). These quotes are not part of the string. They only define the starting and ending of the string. A string may have any character, sign or even space in between them.

For example,

```
>>> str = "Hello Python"
>>> print(str)
Hello Python
```

```

Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:34:40) [MSC v.1927 6
4 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information
>>> str1="Hello Python"
>>> str1
'Hello Python'
>>> str2="12345"
>>> str2
'12345'
>>> str3='Python 3.9.2'
>>> str3
'Python 3.9.2'
Ln: 12 Col: 4

```

CTM: String literals in Python are defined by enclosing text in either type of quotes—single quotes or double quotes. Multiline strings can be represented by using even triple quotes (").

Escape Sequence

Python allows us to represent a string constituting non-graphic characters as well. **Non-graphic characters** are those characters which cannot be directly typed from the keyboard, such as, backspace, tab spaces, carriage return, etc. These non-graphic characters are represented by using escape sequences. An escape sequence is represented by a backslash (\) followed by one or more characters.

It must be remembered that an escape sequence character is represented as a string with one byte of memory and backslash (\) is ignored by the interpreter. The following table (Table 5.1) lists escape sequences supported by Python.

Table 5.1: Escape Sequences in Python

Token	Category
\\	Backslash(\)
\'	Single quote (')
\"	Double quote (")
\a	ASCII Bell (BEL)
\b	ASCII Backspace (BS)
\f	ASCII Formfeed (FF)
\n	ASCII Linefeed (LF)
\r	ASCII Carriage Return (CR)
\t	ASCII Horizontal Tab (TAB)
\v	ASCII Vertical Tab (VT)

- (b) **List:** List is a sequence of items separated by commas and the items are enclosed in square brackets [].

For example, #To create a list

```

>>> list1 = [10, 2.5, "New Delhi", "B45", 66]
#print the elements of the list list1
>>> print(list1)
[10, 2.5, 'New Delhi', 'B45', 66]

```

- (c) **Tuple:** Tuple is a sequence of items separated by commas and items are enclosed in parentheses (). This is unlike list, where values are enclosed in brackets []. Once created, we cannot change the tuple.

For example, #create a tuple tup1

```
>>> tup1 = (100, 60, "Smart phones", 8.7, 's')
# print the elements of the tuple tup1
>>> print(tup1)
(100, 60, 'Smart phones', 8.7, 's')
```

3. **Mapping:** A mapping object maps immutable values to arbitrary objects. This data type is unordered and mutable. Dictionaries in Python fall under Mappings. A dictionary represents data in **key-value** pairs and is accessed using keys, which are immutable. Dictionary is enclosed in curly brackets ({ }).

There is currently only one standard mapping type, the dictionary.

- **Dictionary:** Dictionary in Python holds data items in key-value pairs. Items in a dictionary are enclosed in curly brackets {}. Dictionaries permit faster access to data. Every key is separated from its value using a colon (:) sign.

The key : value pairs of a dictionary can be accessed using the key. The keys are usually strings and their values can be any data type. In order to access any value in the dictionary, we have to specify its key in square brackets [].

For example, #create a dictionary

```
>>> dict = {'Name': 'Ritu', 'Age': 17, 'Class': '12'}
>>> print(dict)
{'Name': 'Ritu', 'Age': 17, 'Class': '12'}
>>> print(dict['Class'])
12
```

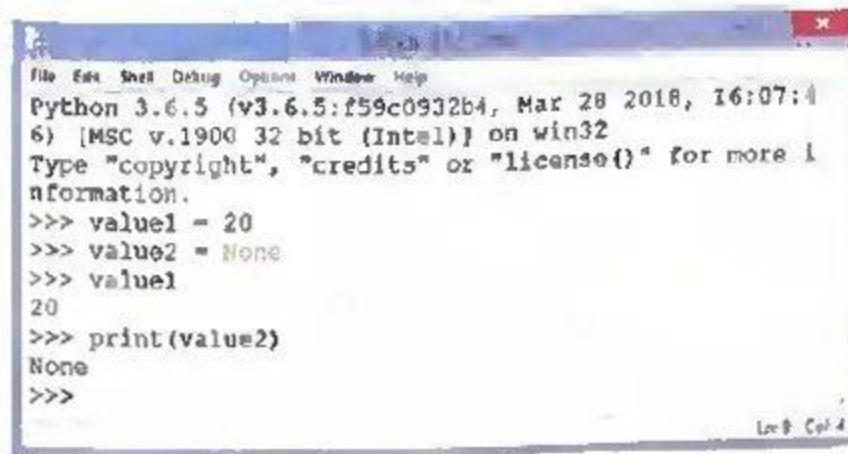
4. **None:** This is a special data type with a single value. It is used to signify the absence of value/condition evaluating to false in a situation. It is represented by **None**. Python doesn't display anything when we give a command to display the value of a variable containing value as None. On the other hand, None will be displayed by printing the value of the variable containing None using print() statement.

For example,

```
>>> value1 = 20
>>> value2 = None
>>> value1
20
>>> value2
>>>
```

Nothing gets displayed. Alternatively, print() statement can be used to display None value as shown below:

```
>>> print(value2)
None
```



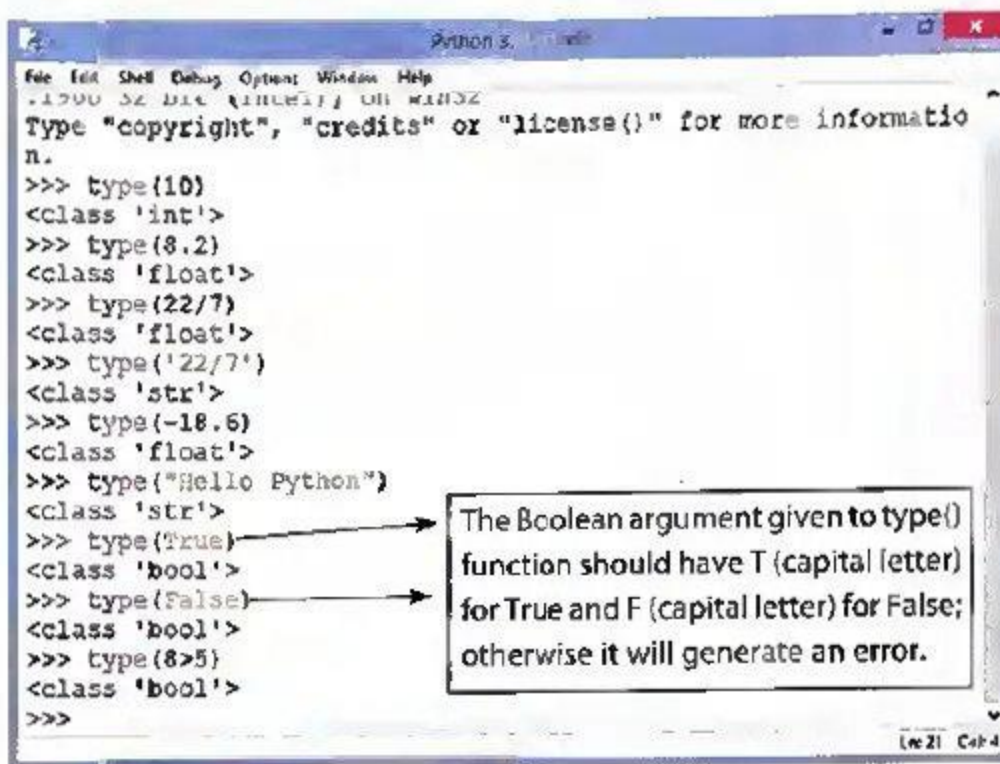
```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> value1 = 20
>>> value2 = None
>>> value1
20
>>> print(value2)
None
>>>
```

Fig. 5.11: Handling None

5.4.1 type()

If you wish to determine the type of a variable, i.e., what type of value does it hold/point to, then `type()` function can be used.

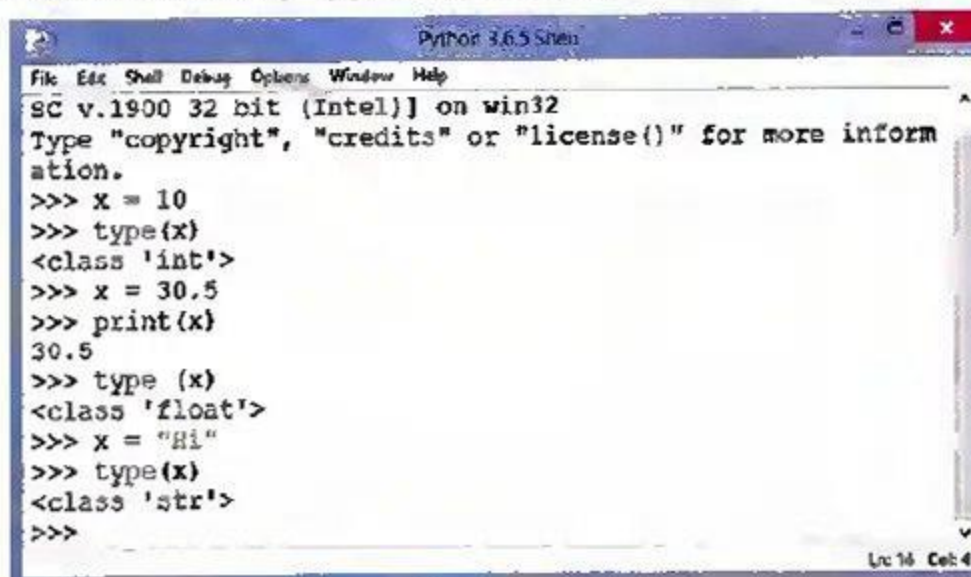
For example,



```
Python 3.6.5 Shell
Type "copyright", "credits" or "license()" for more information.
>>> type(10)
<class 'int'>
>>> type(8.2)
<class 'float'>
>>> type(22/7)
<class 'float'>
>>> type('22/7')
<class 'str'>
>>> type(-18.6)
<class 'float'>
>>> type("Hello Python")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
>>> type(8>5)
<class 'bool'>
>>>
```

The Boolean argument given to `type()` function should have T (capital letter) for True and F (capital letter) for False; otherwise it will generate an error.

The above code can be alternatively typed as follows:



```
Python 3.6.5 Shell
Type "copyright", "credits" or "license()" for more information.
>>> x = 10
>>> type(x)
<class 'int'>
>>> x = 30.5
>>> print(x)
30.5
>>> type(x)
<class 'float'>
>>> x = "Hi"
>>> type(x)
<class 'str'>
>>>
```

Let us take another example for `type()`.

Different types of values can be assigned as under:

```
xyz = "Hello"
w = 18
pi = 3.14159
c = 3.2 + 1.5j
```

Since values have types, so every variable will also have its type. Now we will give the following `type()` statements for the above values and will observe the output for it (Fig. 5.12).

```
type(xyz)
type(w)
type(pi)
type(c)
```

Fig. 5.12: Use of `type()` Function

C) Value of the Variable/Object:

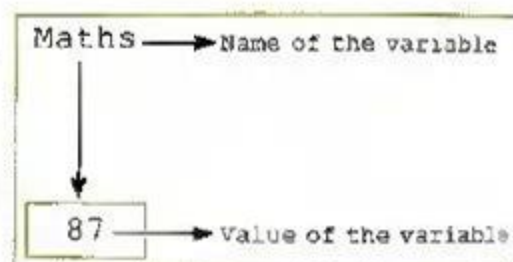
Variables provide a means to name values so that they can be used and manipulated later. To bind value to a variable, we use assignment operator (`=`). This process is also termed as **building of a variable**.

The assignment operator (`=`)

Values are assigned to variables using assignment operator (`=`).

For example, consider the statement,

```
>>> Maths = 87
>>> print(Maths)
```



The statements shall yield the output as shown below:

In the given example, Python associates the name (variable) Maths with the value 87, i.e., the name (variable) Maths is assigned the value 87 or, in other words, the name (variable) Maths refers to value 87. Values are also called objects.

In an assignment statement, the variable that is receiving the value must appear on the left side of the `=` operator, otherwise it will generate an error.

For example, `>>> 87 = Maths` #this is an error

Or we write $x = 10$, which means assigning value 10 to variable x . Similarly, we write another assignment statement as number_1 assigned with value 100, i.e., $\text{number_1} = 100$, as shown in the state diagram given below.



Fig. 5.13: State Diagram of an Assignment Operator

Concept of L-value and R-value:

The L-value represents an object that occupies some identifiable address in memory.

The variable which is at the left hand side of an assignment operator is called the Left value or the L-value. There can be any valid expression on the right-hand side of the assignment operator. The expression which is at the right-hand side of an assignment operator is called the Right value or R-value.

The statement:

$$\text{L-value} = \text{R-value}$$

is called the assignment statement. When the interpreter encounters an assignment statement, it evaluates the right-hand side expression (R-value) and assigns the value to the left-hand side variable (L-value).

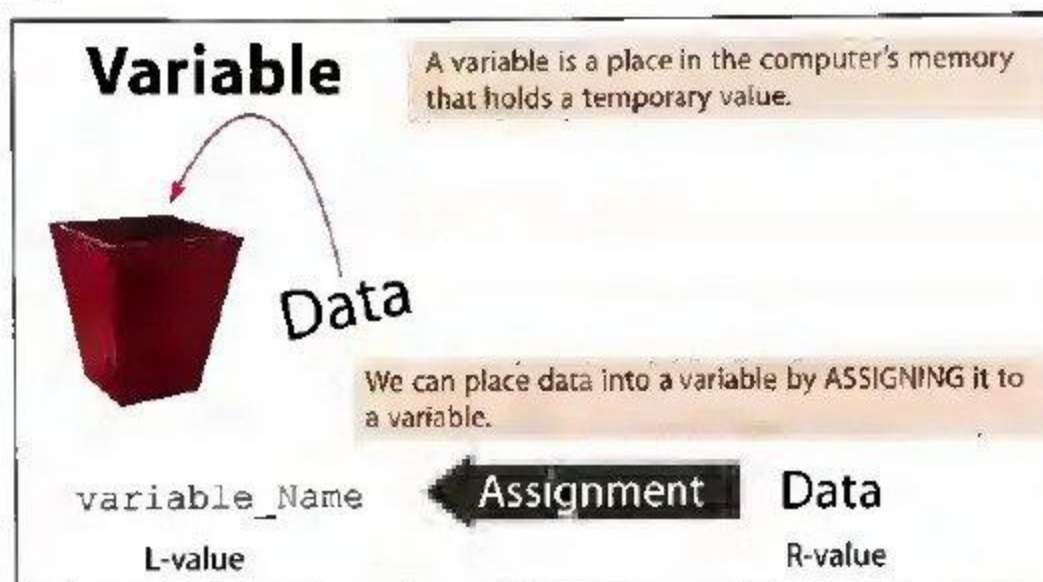


Fig. 5.14: Concept of Assigning a Value to a Variable

For example, suppose we give a statement, $\text{final_value} = 6 + 4 * 2$ in a program. When the interpreter encounters this statement, it will evaluate $6 + 4 * 2$ (i.e., 14), and assign this value to the variable final_value . After this, when we refer to the variable final_value in the program, its value will be 14 until it is changed by another assignment statement.

Let us take a look at a few more examples to have a better understanding of this concept.

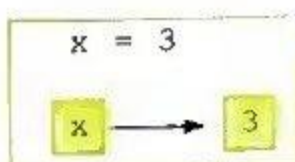


Fig. (i)

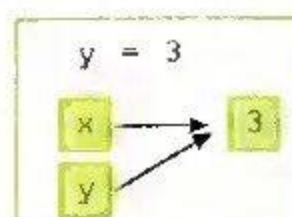


Fig. (ii)

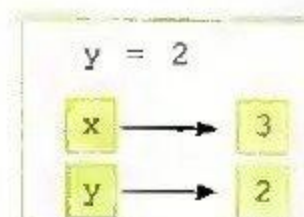


Fig. (iii)

Consider two variables, *x* and *y*, and we type the following assignment statements:

```
x = 3    # variable 'x' is assigned 3 as a value, refer to Fig. (i)
y = 3    # variable 'y' is also assigned value 3, so now y shall be pointing to
         x only, refer to Fig. (ii)
y = 2    # variable 'y' is assigned another value 2, refer to Fig. (iii)
```

Now the final value of *y* becomes 2 and that of *x* remains 3 as assignment operator erases the previous value and assigns new value to the variable '*y*'. Thus, it becomes evident that we can use assignment operator for assigning values to a variable along with their reassignment.

5.4.2 Multiple Assignments

In Python, we can declare multiple variables in a single statement. It is used to enhance the readability of the program.

Multiple assignments in Python can be done in the following two ways:

(i) **Assigning multiple values to multiple variables:**

```
var1, var2, var3, ..., varn = value1, value2, value3, ..., valuen
```

This statement will declare variables *var1*, *var2*, ..., *varn* with values as *value1*, *value2*..., *valuen* respectively.

For example, `msg, day, time = 'Meeting', 'Mon', '9'`

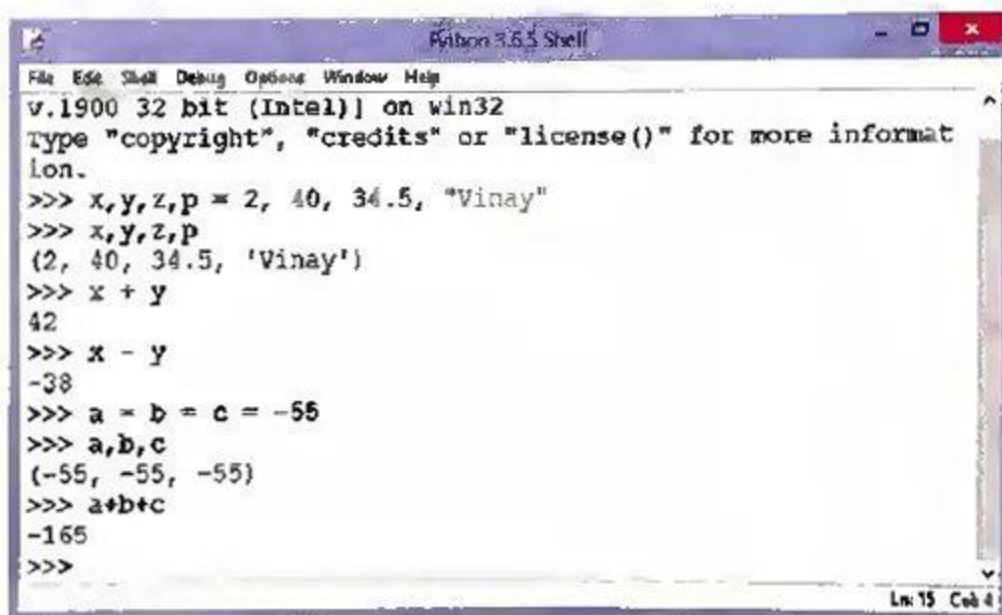
Another statement, `x, y = 2, 3` binds *x* to 2 and *y* to 3.

(ii) **Assigning same value to multiple variables:**

```
var1=var2=var3=...=varn = value1
```

This statement will declare variables *var1*, *var2*, ..., *varn*, all with value as *value1*.

For example, `totalMarks = count = 0`



```
Python 3.5.3 Shell
File Edit Shell Debug Options Window Help
v.1900 32 bit (Intel) on win32
Type "copyright", "credits" or "license()" for more informat
lon.
>>> x,y,z,p = 2, 40, 34.5, "Vinay"
>>> x,y,z,p
(2, 40, 34.5, 'Vinay')
>>> x + y
42
>>> x - y
-38
>>> a = b = c = -55
>>> a,b,c
(-55, -55, -55)
>>> a+b+c
-165
>>>
```

Fig. 5.15: Multiple Assignments in Python

5.4.3 Variable Names

There are certain rules in Python which have to be followed to form valid variable names. Any variable that violates these rules will be an invalid variable name. The rules to be followed are:

- (i) A variable name must start with an alphabet (capital or small) or an underscore character (_).
- (ii) A variable name cannot contain spaces.
- (iii) A variable name can contain any number of letters, digits and underscore (_). No other characters apart from these are allowed.

- (iv) A Python keyword cannot be used as a variable name.
- (v) Variable names are case-sensitive, i.e., name and NAME are two different variables.
- (vi) You should always choose names for your variables that give an indication of what they are used for. In other words, the name should reflect its functionality. For example, a variable that holds the temperature might be named **temperature**, and a variable that holds the speed of a car might be named **speed** instead of giving x and b.

Examples of some valid variable names in Python are:

Unit_per_day, dayofweek, stud_name, father_name, TotalMarks, Age, amount, A24, invoice_no.

Following are examples of some invalid variable names.

Invalid Variable Name	Reason
3dGraph	Name cannot start with a digit
Roll#No	Special symbol (#) is not allowed
First Name	Spaces are not allowed in between
D.O.B	Dots are not allowed
while	Keyword not allowed

Some practical implementations on variables:

Example 1: Write a Python script to input two numbers and find their sum and product.

Solution:

```
n1 = input("Enter first number:")
n1 = eval(n1)
n2 = input("Enter second number:")
n2 = eval(n2)
sum = n1+n2
print("Sum of the numbers=",sum)
prod = n1*n2
print("Product of the numbers=",prod)
```

```
Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:10c0032, Oct 5 2020, 15:31:14) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more (InteractiveShell)
>>>
-- 3637492: C:/Users/User2/AppData/Local/Programs/Python/Python39/python39\python.exe --
Enter first number:10
Enter second number:20
Sum of the numbers= 30
Product of the numbers= 200
In:9 Col:4
```

Example 2: A student has to travel a distance of 450 km by car at a certain average speed. Write Python script to find the total time taken to cover the distance.

Solution:

We know that time taken to cover a distance is calculated as:

$$\text{Time} = \text{Total Distance} / \text{Average Speed}$$

In this problem, we need a data value (average speed) which is not given and neither can it be calculated. So, this value has to be inputted by the user.

Code is:

```
Distance=450
Speed=eval(input("Enter average speed:"))
#to input speed from the user
Time=Distance/Speed #Calculate time taken
print("Distance is:",Distance)
print("Time taken:",Time,"hr")
```

```
Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:10c0032, Oct 5 2020, 15:31:14) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more (InteractiveShell)
>>>
-- 3637492: C:/Users/User2/AppData/Local/Programs/Python/Python39/python.exe --
Enter average speed:60
Distance is: 450
Time taken: 7.5 hr
```

Example 3: Write a Python code to calculate simple interest by inputting the value of Principal amount and rate from the user for a time period of 5 years.

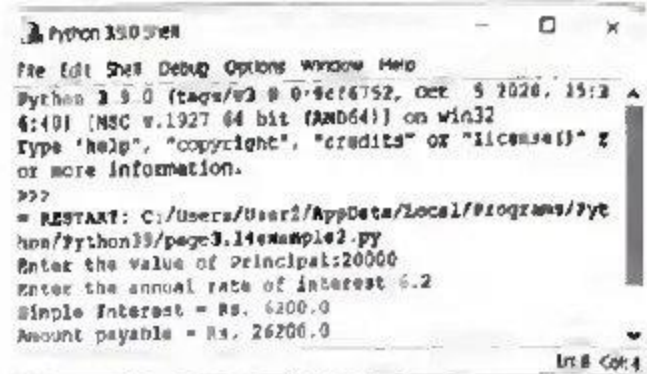
Solution:

Using the formula:

$$\text{Simple Interest} = \text{Principal} \times \text{Rate} \times \text{Time}/100$$

Code:

```
Principal=eval(input("Enter the value of Principal:"))
Rate=eval(input("Enter the annual rate of interest:"))
Time=5
Simple_Int = (Principal*Rate*Time)/100
Amount = Principal+Simple_Int
print("Simple Interest = ₹", Simple_Int)
print("Amount payable = ₹", Amount)
```



Example 4: Write a program to convert the time inputted in minutes into hours and remaining minutes.

Solution:

```
time=int(input("Enter the time in minutes:"))
hours= time//60
mins= time%60
print("Hours are:", hours)
print("Minutes are:", mins)
```

```
Enter the time in minutes:1440
Hours are: 24
Minutes are: 0
```

Note: Here, `eval()` and `int()` functions are used to evaluate the value of a string. It takes a string as an argument, evaluates the string as a number and returns the numeric result. More about `eval()` and `int()` functions has been discussed later in the chapter.

5.5 MUTABLE AND IMMUTABLE DATA TYPES

In certain situations, we may require changing or updating the values of certain variables used in a program. However, for certain data types, Python does not allow us to change the values once a variable of that type has been created and assigned values.

Variables whose values can be changed after they are created and assigned values are called **mutable variables**.

Variables whose values cannot be changed after they are created and assigned values are called **immutable variables**. When an attempt is made to update the value of an immutable variable, the old variable is destroyed and a new variable is created by the same name in memory.

Python data types can be classified into mutable and immutable as under:

- **Examples of mutable objects:** list, dictionary, set, etc.
- **Examples of immutable objects:** int, float, complex, bool, string, tuple, etc. For example, int is an immutable type which, once created, cannot be modified. Consider a variable 'x' of integer type with the initial value 5:

```
>>> x = 5
```

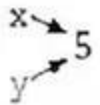
Here, we bind the value to variable named x. Now, we create another variable 'y' which is a copy of variable 'x'.

```
>>> y = x
```

- The above statement will make y refer to value 5 of x, which means you bind the same value 5 to the variable named y.
- We are creating an object of type int. Identifiers x and y point to the same object.

```
x = 5
```

This statement will create a value 5 referenced by x.



```
x → 5  
y → 5
```

Now, we give another statement as:

```
>>> x = x + y
```

- The above statement shall result in adding up the value of x and y and assigning to x. Here, you're re-binding the variable named x to a completely new object, whose value is 5+5=10.

Thus, x gets rebuilt to 10.

```
x    10
```

```
y    5
```

The previous object x with a value of 5 may still exist in computer memory but the binding to it is lost; so there is no variable name available to refer to it anymore.

The object in which x was tagged is changed. Object x = 5 was never modified. An immutable object doesn't allow modification after creation. Another example of immutable object is a string.

```
>>> str = "strings immutable"
```

```
>>> str[0] = 'p'
```

```
>>> print(str)
```

This statement shall result in TypeError on execution. **TypeError: 'str' object does not support item assignment.**

This is because of the fact that strings are immutable. On the contrary, a mutable type object such as a list can be modified even after creation, whenever and wherever required.

```
new_list = [10, 20, 30]
```

```
print(new_list)
```

Output:

```
[10, 20, 30]
```

Suppose we need to change the first element in the above list as:

```
new_list = [10, 20, 30]
```

```
new_list[0] = 100
```

`print(new_list)` will make the necessary updates in the list `new_list` and shall display the output as:

```
[100, 20, 30]
```

This operation is successful since lists are mutable.

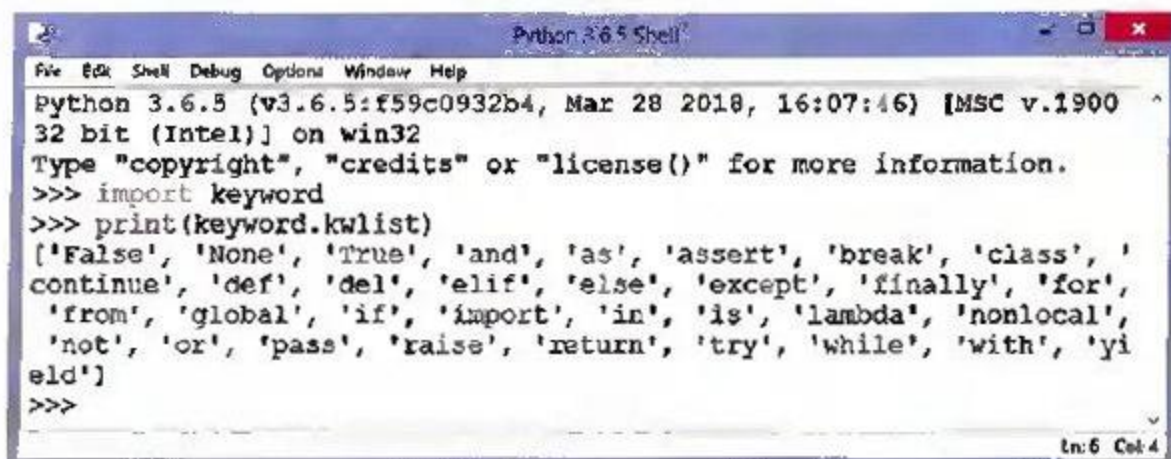
Python handles mutable and immutable objects differently. Immutable objects are quicker to access than mutable objects. Also, immutable objects are fundamentally expensive to "change" because doing so involves creating a copy. Changing mutable objects is cheap.

CTM: Once an immutable object loses its binding to its variable, the Python interpreter may delete the object and release the allocated memory to be used for other tasks. This process of automatic memory management is called Garbage collection.

5.6 KEYWORDS IN PYTHON

Keywords are different from variable names. Keywords are the reserved words used by Python interpreter to recognize the structure of a program. As these words have specific meanings for the interpreter, they cannot be used as variable names or for any other purpose. For checking/displaying the list of keywords available in Python, we have to write the following two statements:

```
import keyword
print(keyword.kwlist)
```



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900
32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> import keyword
>>> print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', '
continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal',
'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yi
eld']
>>>
```

Fig. 5.16: Keywords in Python

CTM: All these keywords are in small alphabets except for `False`, `None` and `True`, which start with capital alphabets.

5.7 EXPRESSIONS

An expression is a combination of value(s), i.e., constant, variable and operators. It generates a single value, which by itself is an expression. Operands contain the values an operator uses and operators are the special symbols which represent simple calculations like addition, subtraction, multiplication, etc.

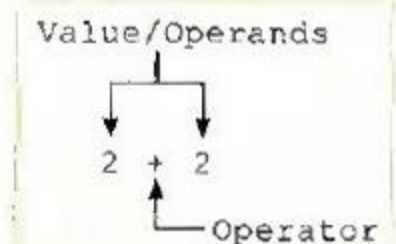


Fig. 5.17: An Expression in Python

Expression	Value
$5 + 2 * 4$	13
$10 / 2 - 3$	2
$8 + 12 * 2 - 4$	28
$6 - 3 * 2 + 7 - 1$	6

➤ Converting Mathematical Expressions to Equivalent Python Expressions

The mathematical expressions that we use in algebra are not used in the same manner in computers. In maths, you use different operators for mathematical calculations. On the other hand, Python as well as other programming languages require different operators for any mathematical operation.

For example,

Algebraic Expression	Operation Being Performed	Programming Expression
6B	6 times B	$6 * B$
(3)(12)	3 times 12	$3 * 12$
4xy	4 times x times y	$4 * x * y$

When converting some algebraic expressions to programming expressions, you may have to insert parentheses that do not appear in the algebraic expression as shown in the table given below:

Algebraic Expression	Python Statement
$y = 3 \left(\frac{x}{2} \right)$	$y = 3 * x / 2$
$z = 3bc + 4$	$z = 3 * b * c + 4$
$a = \frac{x+2}{b-1}$	$a = (x + 2) / (b - 1)$

In all the above examples of expressions, the most important element used is 'Operators', which we will discuss now.

➤ Explicit Conversion

Explicit conversion, also called type casting, happens when data type conversion takes place deliberately, i.e., the programmer forces it in the program. The general form of an explicit data type conversion is:

(new_data_type) (expression)

With explicit type conversion, there is a risk of data loss since we are forcing an expression to be of a specific type.

For example, converting a floating value of $x = 50.75$ into an integer type, i.e., `int(x)`, will discard the fractional part .75 and shall return the value as 50.

```
>>> x = 50.75
>>> print(int(x))
50
```

Following are some of the functions in Python that are used for explicitly converting an expression or a variable into a different type:

Table 5.2: Explicit Type Conversion Functions in Python

Function	Description	Output
<code>int(x)</code>	Converts x into an integer.	<pre>>>>int(10.00) 10</pre>
<code>float(x)</code>	Converts x into a floating-point number.	<pre>>>>float(10) 10.0</pre>
<code>str(x)</code>	Converts x into a string representation.	<pre>>>>str("1000") '1000'</pre>
<code>chr(x)</code>	Converts x into a character.	<pre>>>>chr(65) 'A'</pre>

➤ Implicit Conversion

Implicit conversion, also known as coercion, happens when data type conversion is done automatically by Python and is not instructed by the programmer.

Example 5: Program to illustrate implicit type conversion from int to float.

```

#Implicit type conversion from int to float
num1 = 10 #num1 is an integer
num2 = 55.0 #num2 is a float
sum1 = num1 + num2 #sum1 is sum of a float and an integer
print(sum1)
print(type(sum1))

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_implicit_conv1.py
65.0
<class 'float'>
>>>

```

In the above example, an integer value stored in variable `num1` is added to a float value stored in variable `num2`, and the result gets automatically converted into a float value stored in variable `sum1` without explicitly telling the system. This is an example of implicit data conversion.

The reason for the float value not being converted into an integer instead is due to type promotion that allows performing operations (whenever possible) by converting data into a wider-sized data type without any loss of information.

Example 6: Write a Python code to calculate simple interest and amount payable by inputting the value of principal amount and rate from the user for a time period of 5 years.

(Formula used: $\text{Simple Interest} = \text{Principal} * \text{Rate} * \text{Time}/100$)

```

principal=eval(input("Enter the value of Principal:"))
Rate=eval(input("Enter the annual rate of interest:"))
Time=5
Simple_Int = Principal*Rate*Time/100
Amount = Principal+Simple_Int
print("Simple Interest = ",Simple_Int)
print("Amount payable = ",Amount)

Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46)
[MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more inf
ormation.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python
/Python36-32/prog_si_class12.py
Enter the value of Principal:1000
Enter the annual rate of interest:15
Simple Interest =  6000.0
Amount payable =  14000.0
>>>

```

5.8 OPERATORS

Operators are special symbols which represent computation. They are applied on operand(s), which can be values or variables. Same operator can behave differently on different data types. Operators when applied on operands form an expression.

Operators are categorized as Arithmetic, Relational, Logical and Assignment. Value and variables, when used with operators, are known as operands.

5.8.1 Mathematical/Arithmetic Operators

A number of operators are available in Python to form and solve arithmetic or algebraic expressions.

Table 5.3: Arithmetic Operators in Python

Symbol	Description	Example 1	Example 2
+	Addition	>>> 55+45 100	>>> 'Good' + 'Morning' GoodMorning
-	Subtraction	>>> 55-45 10	>>> 30-80 -50
*	Multiplication	>>> 55*45 2475	>>> 'Good'*3 GoodGoodGood
/	Division	>>> 17/5 3.4 >>> 17/5.0 3.4 >>> 17.0/5 3.4	>>> 28/3 9.3333333333333334
%	Remainder/ Modulo	>>> 17%5 2	>>> 23%2 1
**	Exponentiation	>>> 2**3 8 >>> 16**0.5 4.0	>>> 2**8 256
//	Integer Division (Floor Division)	>>> 7.0//2 3.0	>>> 3//2 1 >>> -9//2 -5 (in case of negative digit, the result is floored)

For example,

```
print("18 + 5 =", 18 + 5)      #Addition
print("18 - 5 =", 18 - 5)      #Subtraction
print("18 * 5 =", 18 * 5)      #Multiplication
print("27 / 5 =", 27 / 5)      #Division
print("27 // 5 =", 27 // 5)    #Integer Division
print("27 % 5 =", 27 % 5)      #Modulus
print("2 ** 3 =", 2 ** 3)      #Exponentiation
print("-2 ** 3 =", -2 ** 3)    #Exponentiation
```

Output:

```

18 + 5 = 23
18 - 5 = 13
18 * 5 = 90
27 / 5 = 5.4
27 // 5 = 5
27 % 5 = 2
2 ** 3 = 8
-2 ** 3 = -8

```

Concatenating Strings

Strings can be added together with the plus (+) operator. To concatenate the string "Hello Python", give the statement using concatenation operator (+):

```
>>> "Hello" + "Python"
```

Output:

```

'HelloPython'
>>> print("'how' + 'are' + 'you?':", 'how' + 'are' + 'you?')

```

Output:

```
'how' + 'are' + 'you?': howareyou?
```

Similarly, entering "*****" * 5 will yield:

```

*****
>>> print("'hello' * 5 :", 'hello' * 5)

```

Output:

```
'hello' * 5 : hellohellohellohellohello
```

Unary and Binary operators: An operator can be termed as unary or binary depending upon the number of operands it takes. A unary operator takes only one operand and a binary operator takes two operands. For example, in the expression $-6 * 2 + 8 - 3$, the first minus sign is a unary minus and the second minus sign is a binary minus. The multiplication and addition operators in the above expression are binary operators.

Precedence of Arithmetic Operators

() (parentheses)

** (exponentiation)

- (negation)

/ (division) // (integer division) * (multiplication) % (modulus)

+ (addition) - (subtraction)

↓ decreasing order

More Examples of Arithmetic Operators

Evaluate the following expressions:

(1) $12 + 3 * 4 - 6/2$

(2) $(12 + 3) * 4 - 6/2$

(3) $12 + 3 * (4 - 6) / 2$

(4) $12 + (3 ** 4 - 6)/2$

(5) $12 * (3 \% 4) // 2 + 6$

(6) $12 \% 3 ** 4 // 5 + 6$

Solution:

$$\begin{aligned}
 (1) \quad & 12 + 3 * 4 - 6 / 2 \\
 & = 12 + 12 - 6 / 2 \\
 & = 12 + 12 - 3.0 \\
 & = 24 - 3.0 \\
 & = 21.0
 \end{aligned}$$

$$\begin{aligned}
 (2) \quad & (12 + 3) * 4 - 6 / 2 \\
 & = 15 * 4 - 6 / 2 \\
 & = 60 - 6 / 2 \\
 & = 60 - 3.0 \\
 & = 57.0
 \end{aligned}$$

$$\begin{aligned}
 (3) \quad & 12 + 3 * (4 - 6) / 2 \\
 & = 12 + 3 * (-2) / 2 \\
 & = 12 + (-6) / 2 \\
 & = 12 - 3.0 \\
 & = 9.0
 \end{aligned}$$

$$\begin{aligned}
 (4) \quad & 12 + (3 ** 4 - 6) / 2 \\
 & = 12 + (81 - 6) / 2 \\
 & = 12 + 75 / 2 \\
 & = 12 + 37.5 \\
 & = 49.5
 \end{aligned}$$

$$\begin{aligned}
 (5) \quad & 12 * (3 \% 4) // 2 + 6 \\
 & = 12 * 3 // 2 + 6 \\
 & = 36 // 2 + 6 \\
 & = 18 + 6 \\
 & = 24
 \end{aligned}$$

$$\begin{aligned}
 (6) \quad & 12 \% 3 ** 4 // 5 + 6 \\
 & = 12 \% 81 // 5 + 6 \\
 & = 12 // 5 + 6 \\
 & = 2 + 6 \\
 & = 8
 \end{aligned}$$

5.8.2 Relational Operators

Relational operators are used for comparing two expressions and result in either True or False. In an expression comprising both arithmetic and relational operators, the arithmetic operators have higher precedence than the relational operators.

Syntax:

<expression1> <comparison operator> <expression2>

Table 5.4: Relational Operators in Python

Symbol	Description	Example 1	Example 2
<	less than	<pre>>>> 7<10 True >>> 7<5 False >>> 7<10<15 True >>> 7<10 and 10<15 True</pre>	<pre>>>> 'Hello'<'Goodbye' False >>> 'Goodbye'<'Hello' True</pre>
>	greater than	<pre>>>> 7>5 True >>> 10>10 False</pre>	<pre>>>> 'Hello'>'Goodbye' True >>> 'Goodbye'>'Hello' False</pre>
<=	less than equal to	<pre>>>> 2<=5 True >>> 7<=4 False</pre>	<pre>>>> 'Hello'<='Goodbye' False >>> 'Goodbye'<='Hello' True</pre>
>=	greater than equal to	<pre>>>> 10>=10 True >>> 10>=12 False</pre>	<pre>>>> 'Hello'>='Goodbye' True >>> 'Goodbye'>='Hello' False</pre>
!=, <>	not equal to	<pre>>>> 10!=11 True >>> 10!=10 False</pre>	<pre>>>> 'Hello'!='HELLO' True >>> 'Hello'!='Hello' False</pre>
==	equal to	<pre>>>> 10==10 True >>> 10==11 False</pre>	<pre>>>> 'Hello'=='Hello' True >>> 'Hello'=='Good Bye' False</pre>

For example,

```
>>> print("23 < 25 :", 23 < 25)           #less than
>>> print("23 > 25 :", 23 > 25)           #greater than
>>> print("23 <= 23 :", 23 <= 23)         #less than or equal to
>>> print("23 - 2.5 >= 5 * 4 :", 23 - 2.5 >= 5 * 4) #greater than or equal to
>>> print("23 == 25 :", 23 == 25)         #equal to
>>> print("23 != 25 :", 23 != 25)         #not equal to
```

Output:

```
23 < 25 : True
23 > 25 : False
23 <= 23 : True
23 - 2.5 >= 5 * 4 : True
23 == 25 : False
23 != 25 : True
```

- When the relational operators are applied to strings, strings are compared left to right, character by character, based on their ASCII codes, and also called ASCII values.

```
print("'hello' < 'Hello' :", 'hello' < 'Hello')
print("'hi' > 'hello' :", 'hi' > 'hello')
```

Output:

```
'hello' < 'Hello' : False
'hi' > 'hello' : True
```

Python starts by comparing the first element from each sequence. If they are equal, it goes on to the next element, and so on, until it finds elements that differ. Subsequent elements are not considered (even if they are greater).

5.8.3 Logical Operators

The three logical operators supported by Python are **and**, **or** and **not**. The logical operator evaluates to either True or False based on the logical operands on either side. Every value is logically either True or False. Logical operators are also used to combine relational conditions.

Symbol	Description
or	If any one of the operands is true, then the condition becomes true.
and	If both the operands are true, then the condition becomes true.
not	Reverses the state of operand/condition.

For example,

```
>>> print("not True < 25 :", not True)     #not operator
>>> print("10 < 25 and 5 > 6 :", 10 < 25 and 5 > 6) #and operator
>>> print("10 < 25 or 5 > 6 :", 10 < 25 or 5 > 6)  #or operator
```

Precedence of Logical Operators

```
not
and
or
```

Decreasing
order

CTM: Relational operators are also known as Comparison operators and yield values, True or False, as the output.

5.8.4 Shorthand/Augmented Assignment Operators

A Shorthand Assignment Operator (or compound assignment operator or an augmented operator) is a combination of a binary operation and assignment. Different augmented assignment operators available in Python are as follows:

Note: We assume the value of variable `x` as 12 for a better understanding of these operators.

<code>+=</code>	added and assign back the result to left operand	<code>x+=2</code>	The operand/expression/constant written on RHS of operator will change the value of <code>x</code> to 14
<code>-=</code>	subtracted and assign back the result to left operand	<code>x-=2</code>	<code>x</code> will become 10
<code>*=</code>	multiplied and assign back the result to left operand	<code>x*=2</code>	<code>x</code> will become 24
<code>/=</code>	divided and assign back the result to left operand	<code>x/=2</code>	<code>x</code> will become 6
<code>%=</code>	taken modulus using two operands and assign the result to left operand	<code>x%=2</code>	<code>x</code> will become 0
<code>**=</code>	performed exponential (power) calculation on operators and assign value to the left operand	<code>x**=2</code>	<code>x</code> will become 144
<code>//=</code>	performed floor division on operators and assign value to the left operand	<code>x//=2</code>	<code>x</code> will become 6

For example,

Shorthand Notation:

`a = a <operator> b` is equivalent to
`a <operator>= b`

```
a = 6
a = a + 5
print(a)
a = 6
a += 5
print(a)
```

Output:

```
11
11
```

Learning Tips:

1. Same operator may perform a different function depending on the data type of the value to which it is applied.
2. Division operator `/` behaves differently on integer and float values.

5.8.5 Membership Operators

Python offers two membership operators for checking whether a particular character exists in the given string or not. These operators are `'in'` and `'not in'`.

'in' operator: It returns `True` if a character/substring exists in the given string.

'not in' operator: It returns `True` if a character/substring does not exist in the given string.

To use membership operator in strings, it is required that both the operands used should be of string type, i.e.,

`<substring> in <string>`

`<substring> not in <string>`

```

Python 2.7.10 Shell
File Edit Shell Debug Options Window Help
>>> 'H' in 'Hello'
True
>>> 'y' in 'Hello'
False
>>> 'HEL' in 'Hello'
False
>>> 'y' not in 'Hello'
True
>>> 'H' not in 'Hello'
False

>>> str1="my"
>>> string='my book'
>>> str1 == string
True

```

Python being case-sensitive

5.8.6 Identity Operators

Identity operators are used to compare the objects if both the operands are actually the same objects and share the same memory location. These operators are 'is' and 'is not'.

'is' operator: It returns True if both variables are pointing to the same object.

'is not' operator: It returns True if both variables are not the same objects.

For example,

```

>>> P=100
>>> Q=P
>>> P is Q
True
>>> id(P)
2160571667920
>>> id(Q)
2160571667920
>>> R=500
>>> R is P
False
>>> R is not P
True
>>>

```

P and Q pointing to the same object

R and P are not pointing to same object

5.9 USER INPUT

While writing a program, we need to fetch some input from the user. The values inputted by the user are fetched and stored in the variables. Python provides three important functions for getting user's input.

1. **input():** input() function is used to get data from the user while working with the script mode. It enables us to accept an input in the form of string from the user without evaluating its value. The function input continues to read input text from the user until it encounters a new line.

Let us see a simple example to fetch user's name and to display "Welcome" message concatenated with user's name. The input() function takes String as an argument. During execution, input() shows the prompt to the user and waits for the user to input a value from the keyboard. When the user enters value from the keyboard, input() returns this value to the script mode. In almost all the cases, we store this value in a variable.

When the first statement of the above example is executed, the user is prompted to enter his/her name. The name entered by the user is stored in the variable name. After this, the variable name gets combined with the 'Welcome' message and can be used as many times as needed.

```

prog2_chap4.py - C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/Python36-32
File Edit Shell Debug Options Window Help
name = input('Enter your Name :')
print('Welcome', name)

```

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/prog2_chap4.py
Enter your Name : Sonia
Welcome Sonia
>>>

```

For example, the program given below is for adding the two values inputted by the user through input() function.

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> num_1 = input("Enter first number: ")
Enter first number: 30
>>> num_2 = input("Enter second number: ")
Enter second number: 40
>>> result = num_1 + num_2
>>> print("Sum is: ", result)
Sum is: 3040
>>>

```

As shown in the above figure, we are getting unexpected output on execution. As per the program, the expected output was 70, after adding the two inputted numbers 30 + 40. But instead we are getting 3040, which signifies that 30 and 40 are fetched as strings and not as numeric value and hence + will act as concatenation operator instead of mathematical addition operator.

```

prog3_chap4.py - C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/Python36-32
File Edit Shell Debug Options Window Help
num_1 = int(input("Enter first number: "))
num_2 = int(input("Enter second number: "))
result = num_1 + num_2
print("Sum is: ", result)

```

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/prog3_chap4.py
Enter first number: 30
Enter second number: 40
Sum is : 70
>>>

```

Hence, we need to modify our program using another function int().

int() function converts the inputted string value into numeric value and shall store the value 30 as a number and not as a string. Thus, we will obtain the desired result as 70 (sum of 30 and 40).

2. **eval()**: This function is used to evaluate the value of a string. It takes a string as an argument, evaluates this string as a number, and returns the numeric result.

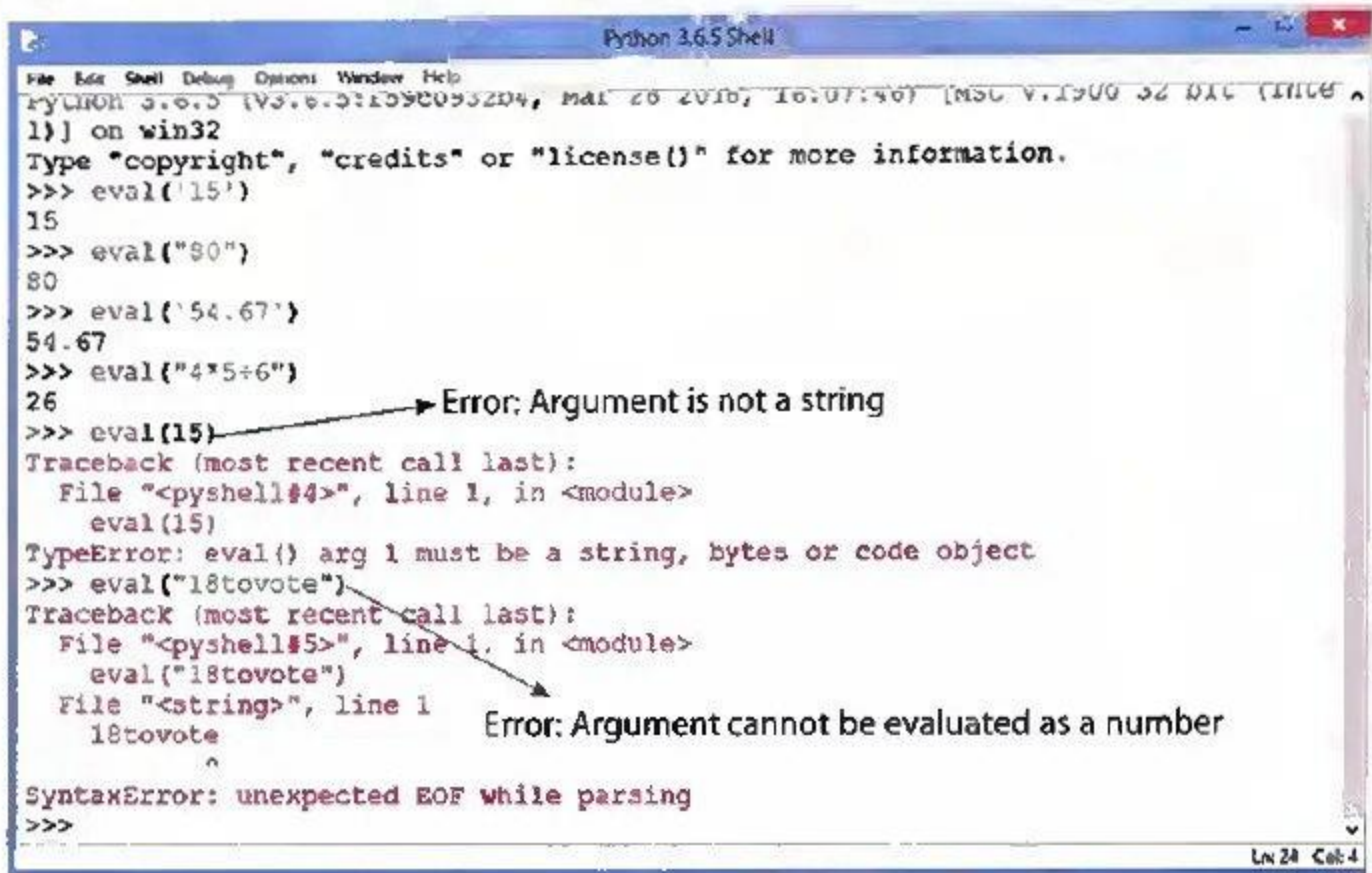
If the given argument is not a string, or if it cannot be evaluated as a number, then `eval()` results in an error.

For example,

```
a = eval('15+10')
print(a)
```

Output:

25



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (tags/v3.6.5:13b003204, Mar 26 2018, 16:07:56) [MSO V.1.000 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> eval('15')
15
>>> eval("80")
80
>>> eval('54.67')
54.67
>>> eval("4*5+6")
26
>>> eval(15)
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    eval(15)
TypeError: eval() arg 1 must be a string, bytes or code object
>>> eval("18tovote")
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    eval("18tovote")
  File "<string>", line 1
    18tovote
    ^
SyntaxError: unexpected EOF while parsing
>>>
```

In all of the above programs, we have used built-in functions available in Python library for performing numeric and string operations.

Let us discuss another way of writing codes using User-defined Functions.

5.10 USER-DEFINED FUNCTIONS

A **function** is a group of statements that exists within a program for the purpose of performing a specific task. Instead of writing a large program as one long sequence of instructions, it can be written as several small functions, each one performing a specific part of the task.

➤ How to Define and Call a Function in Python

A user-defined Python function is created or defined by the `def` statement followed by the function name and parentheses `(( ))` as shown in the syntax given below:

Syntax:

```
def function_name(comma_separated_list_of_parameters):
    """docstring"""
    statement(s)
```

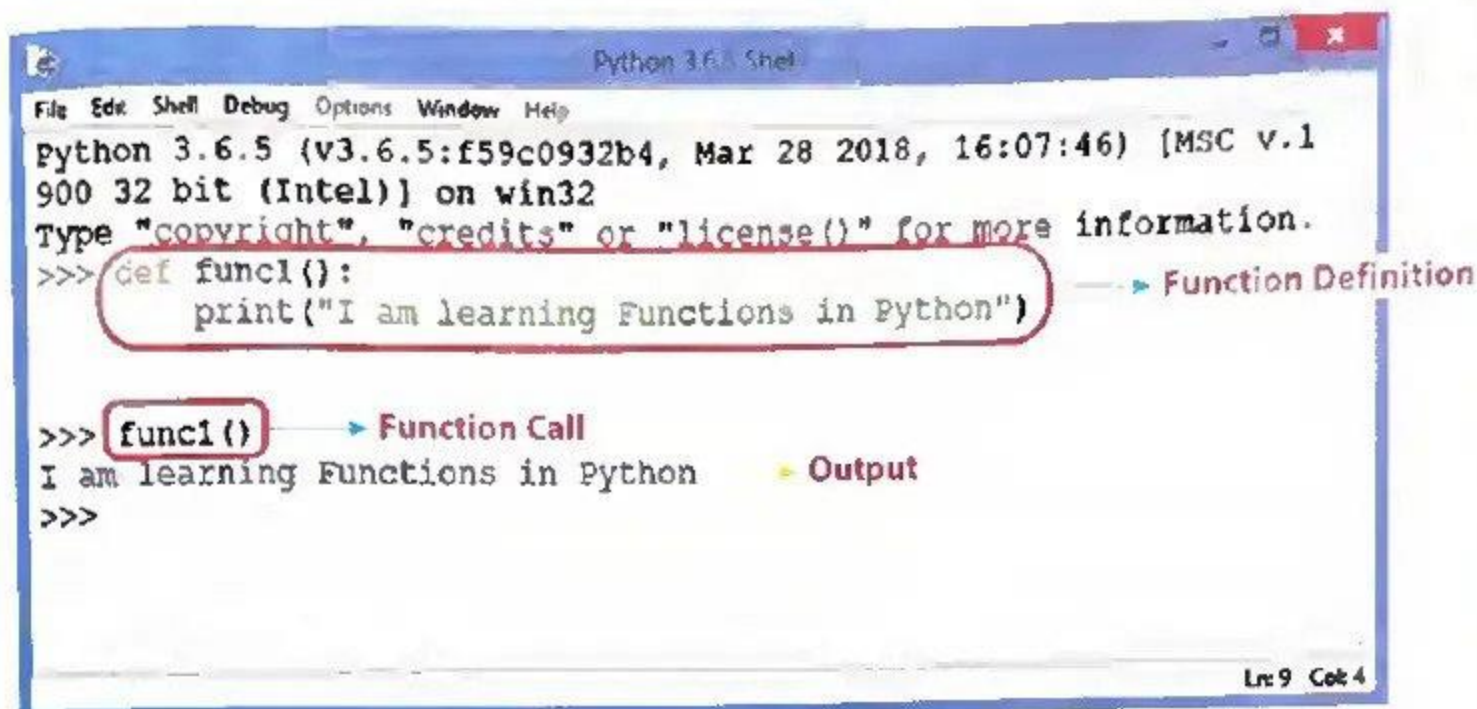
Keyword Function Definition

POINT TO REMEMBER

Statements below `def` begin with four spaces. This is called indentation. It is a requirement of Python that the code following a colon must be indented.

Practical Implementation-1

Let us define a simple function (which does not return any value) by using the command "`def func1():`" and call the function. The output of the function will be "I am learning Functions in Python" (Fig. 5.18).



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1
900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> def func1():
    print("I am learning Functions in Python")
>>> func1()
I am learning Functions in Python
```

Annotations in the image:

- `def func1():` is labeled "Function Definition".
- `func1()` is labeled "Function Call".
- The output `I am learning Functions in Python` is labeled "Output".

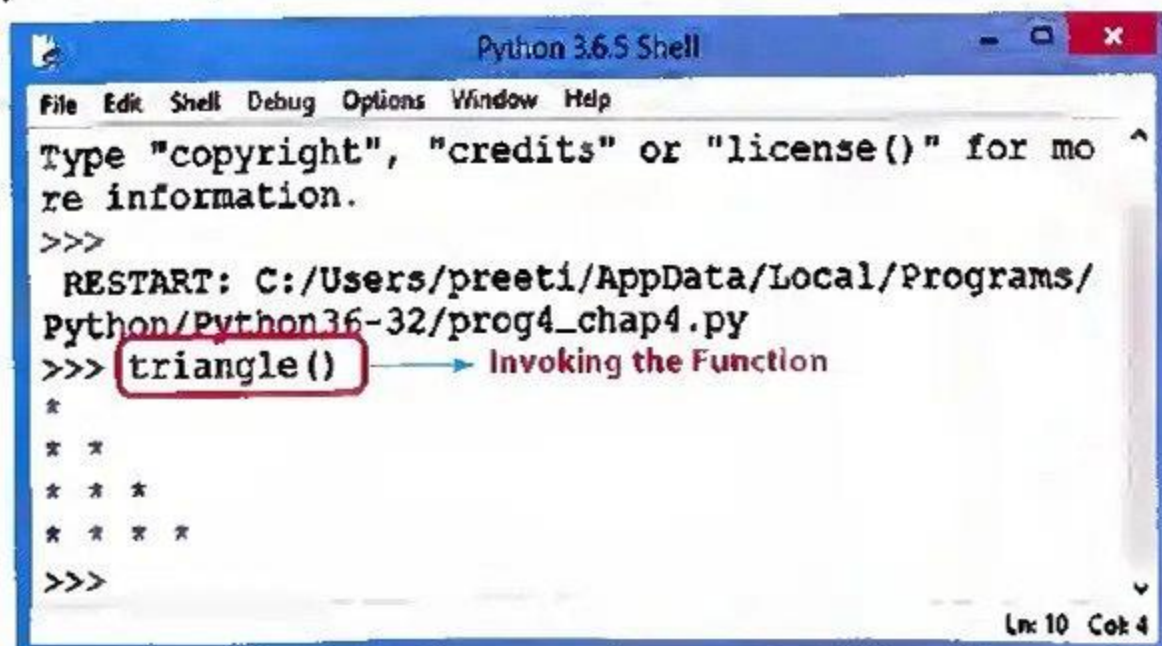
Fig. 5.18: Function Definition (Interactive Mode)

Practical Implementation-2

To write a user-defined function to print a right-angled triangle.



```
prog4_chap2.py - C:/Users/preeti/AppData/Local/Pr...
File Edit Format Run Options Window Help
def triangle():
    print("*")
    print("* *")
    print("* * *")
    print("* * * *")
```



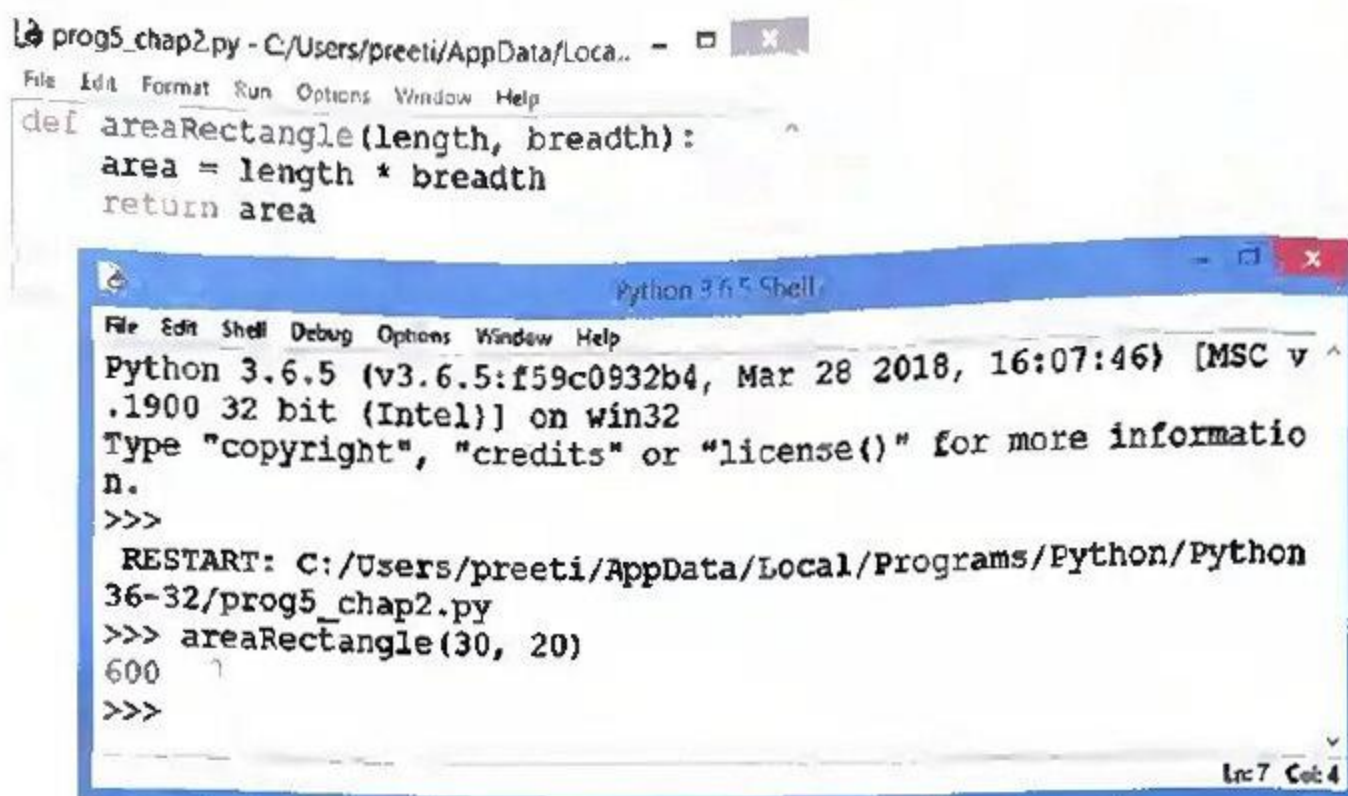
```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Type "copyright", "credits" or "license()" for more information.
>>> RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/prog4_chap4.py
>>> triangle()
*
* *
* * *
* * * *
>>>
```

Annotation in the image:

- `triangle()` is labeled "Invoking the Function".

Practical Implementation-3

To write a Python function to compute area of a rectangle.



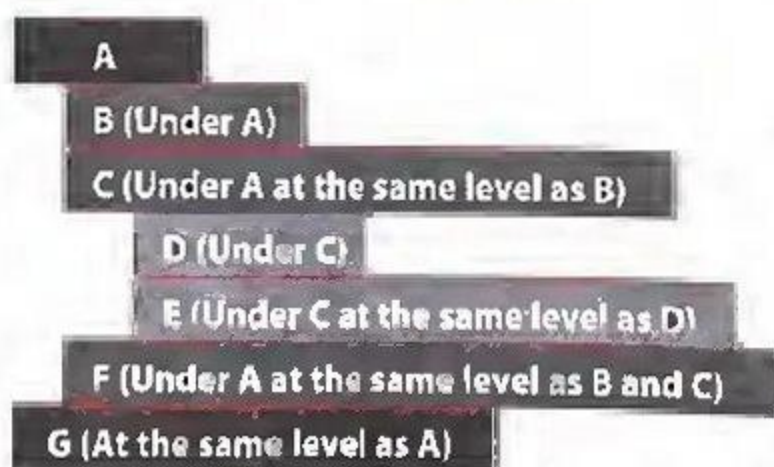
```
prog5_chap2.py - C:/Users/preeti/AppData/Local...  
File Edit Format Run Options Window Help  
def areaRectangle(length, breadth):  
    area = length * breadth  
    return area  
  
Python 3.6.5 Shell  
File Edit Shell Debug Options Window Help  
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v ^  
.1900 32 bit (Intel)] on win32  
Type "copyright", "credits" or "license()" for more informatio  
n.  
>>>  
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python  
36-32/prog5_chap2.py  
>>> areaRectangle(30, 20)  
600  
>>>
```

In the above example, we have given the return statement:

`return area`, which will return back the value of area of rectangle calculated to the function call `areaRectangle()` along with `length` and `breadth` as the arguments to function `areaRectangle()`.

5.11 INDENTATION IN PYTHON

Python follows a particular style of indentation to define the code, since Python functions don't have any explicit beginning or end, like curly braces, to indicate the start and stop for the function; they have to rely on this indentation. Indentation is shifting of a statement to the right of another statement by adding white space before a statement. In Python, indentation is used to form suites (blocks of statements) that indicates the group of statements belongs to a particular block of code. A suite is a group of statements which is treated as a unit. If a suite (let us call it B) is indented with respect to its above statement (let us call it A), then execution of suite B depends upon the execution of statement A. This concept is used extensively in functions, conditional statements and loops (covered later). The concept of indentation can be understood better with the help of the following figure:



Here we take a simple example with "print" command.

Practical Implementation-4

When we write "print" function right below the def func1(): It will show "indentation error: expected an indented block".

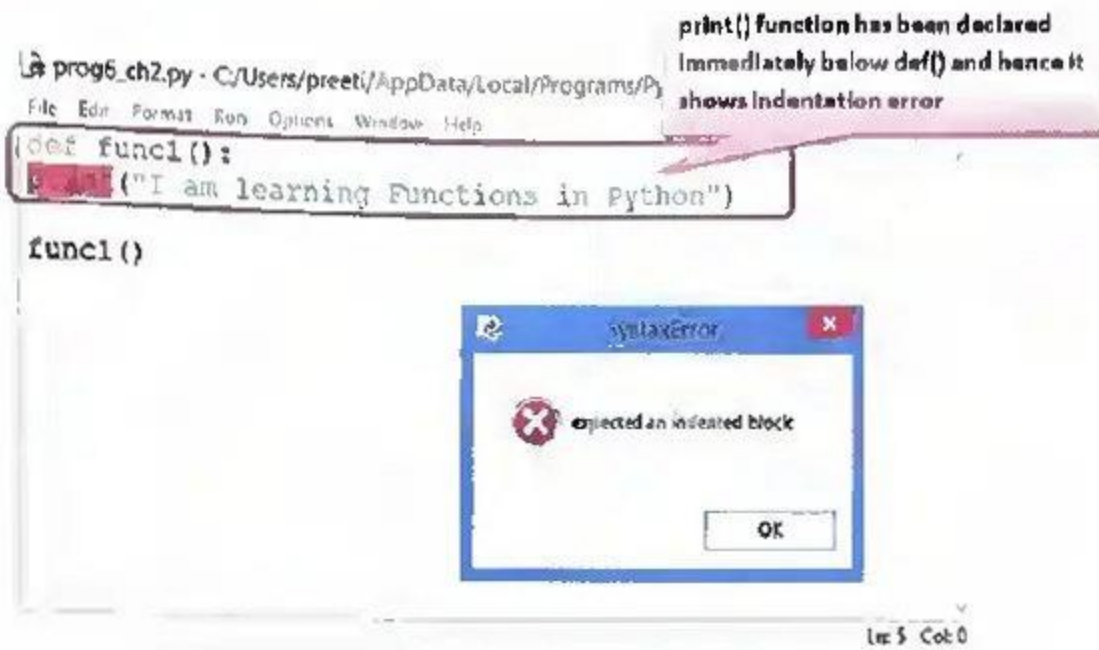


Fig. 5.19(a): Improper Indentation Results in Error Generation

The above error can be removed by making proper indentation before the `print()` function. At least one indent is required to make your code work successfully.

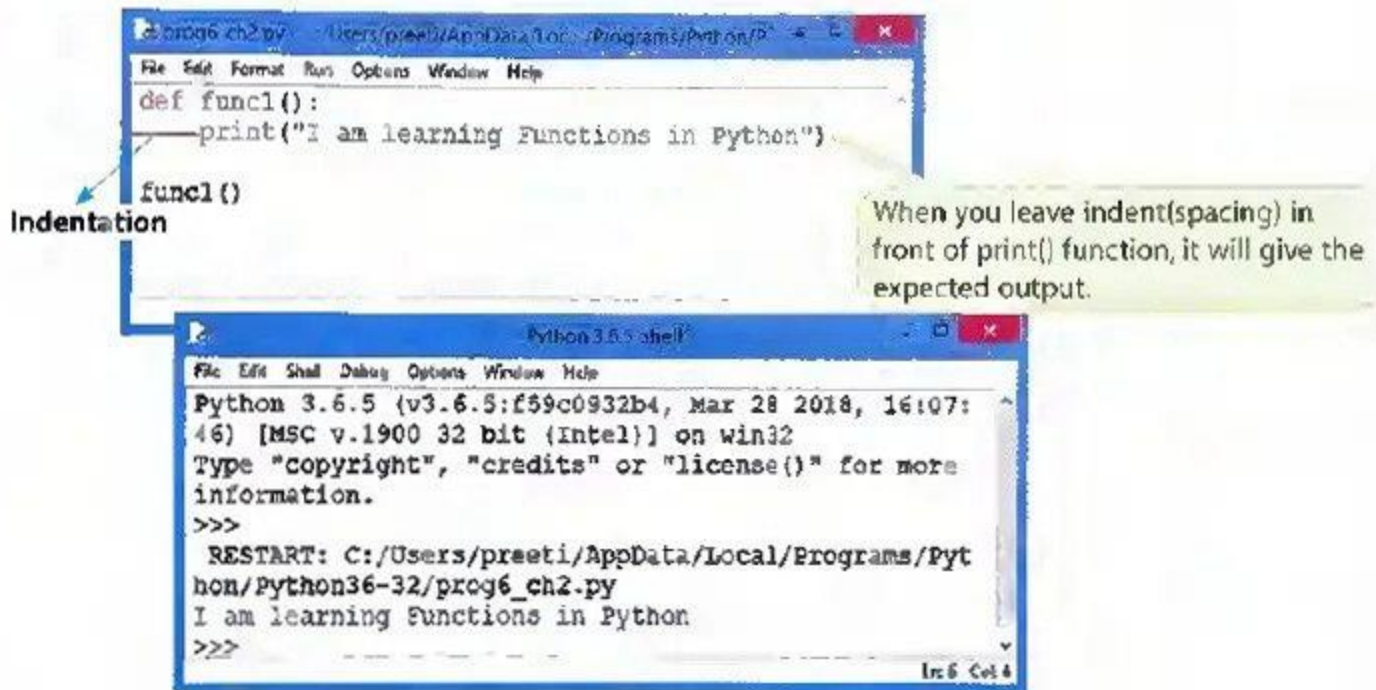


Fig. 5.19(b): Significance of Indentation

5.12 RULES AND CONVENTIONS FOR WRITING PYTHON PROGRAMS

While working with Python, the following rules and conventions are to be kept in mind:

- Statement Termination:** There is no demarcation or symbol in Python to indicate the termination of a statement. When you end typing a statement in Python by pressing Enter key, the statement is considered terminated by default.
- Clarity and Simplification of Expression:** An expression in a program is a sequence of operators and operands to do an arithmetic or logical computation. Some of the valid expressions are:
 - Assigning a value to a variable.
 - Performing mathematical calculations using one or more variables.
 - Comparing two values.

Writing unambiguous expressions is a skill that must be developed by every programmer. Following are the guidelines to be kept in mind while writing such expressions:

- Avoid the usage of expressions that give ambiguous results, creating confusion.
- Avoid using complex expressions.
- c) **Simplicity of Instructions:** The instructions given to a computer must be clear and simple so that any user reading those instructions should be able to understand them.

It has been observed that sometimes even the programmers fail to understand the logic of their own programs after some time. As a result, the maintenance and modification of such programs would become very difficult. Following are a few tips that should be kept in mind while writing the instruction set:

- Avoid complex instructions.
- One instruction per task should be given.
- Follow the standards of a language.
- Use appropriate comments.
- Keep the names of the variables short, simple and meaningful.
- d) **Maximum Line Length:** Line length should be of maximum 79 characters.
- e) **Lines and Indentation:** Blocks of code in Python are denoted by line indentation, which is implemented using 4 spaces per indentation level.
- f) **Avoid multiple statements on one line:** Although you can type multiple statements using semicolon (;) between two statements in a single line, it is not recommended.

5.13 COMMENTS

Comments are statements in a script that are used to explain Python code and are ignored by the Python interpreter and, therefore, have no effect on the actual output of the code. Comments make the code more readable and understandable for human beings. This makes the revision/modification of the code and easier maintenance as well as finding bugs faster. Comments are not necessary in a script, but these are highly recommended if the script is too complex and if it is written by multiple programmer to enhance the readability and understanding of the code.

A comment in Python starts with a hash symbol (#) anywhere in a line and extends till the end of the line. The following code contains two single line comments:

```
# This is a single line comment
x=10
y=x+100 # This is inline comment
print(y)
```

Since Python will ignore string literals that are not assigned to a variable. So, we can use triple quotes for specifying a multiline comment. For example, "This is multiline comment."

5.14 DEBUGGING

Apart from computer programs that we code, an exception or error or unusual condition can occur even in our daily life. Consider a real-life scenario—you leave your house for office by car. On the way, the tyre bursts and you are forced to stop the car and replace the tyre. So, in this scenario, the punctured tyre is the exception which has occurred unexpectedly. In such situations, we are required to carry along a spare tyre so that we can change the punctured tyre then and there and continue with our journey, which is described as exception handling.



Likewise, when we are browsing a web page and that web page is unreachable because there is no network connection or that there is no microphone available to create an audio recording—these are examples of exceptions or errors.

Sometimes an application can recover from an error and still provide normal, expected behaviour, sometimes errors get reported to the user, sometimes they get logged to a file, etc. It depends on the specific error and the application. Same is applicable in the field of computer programming also.

Most application codes written for developing applications have errors in them. When your application suddenly freezes for no apparent reason, it is usually because of an error. But we all understand that programs have to deal with errors.

The process of finding errors in a program is termed as Debugging.

Due to errors, a program may not execute or may generate wrong output. So, it becomes necessary to find out and remove the errors for successful execution of a program. Errors in Python are classified mainly into three types:

- (a) Syntax Error
- (b) Run-time Error
- (c) Logical Errors

5.14.1 Syntax Error

A syntax error is an error in the syntax of a sequence of characters or tokens that is intended to be written in a particular programming language. These types of errors are generated when we violate the syntax or, in other words, the grammatical rules of a programming language. Syntax errors are the most common type of errors which are easily traceable. Some examples of Python Syntax errors are:

1. Incorrect indentation
2. Misspelling a keyword
3. Leaving out a symbol such as colon(:), comma(,) or parentheses(()).

They can be corrected by the user as the reason for the error and an appropriate message about what is wrong in the program is displayed.

For example,

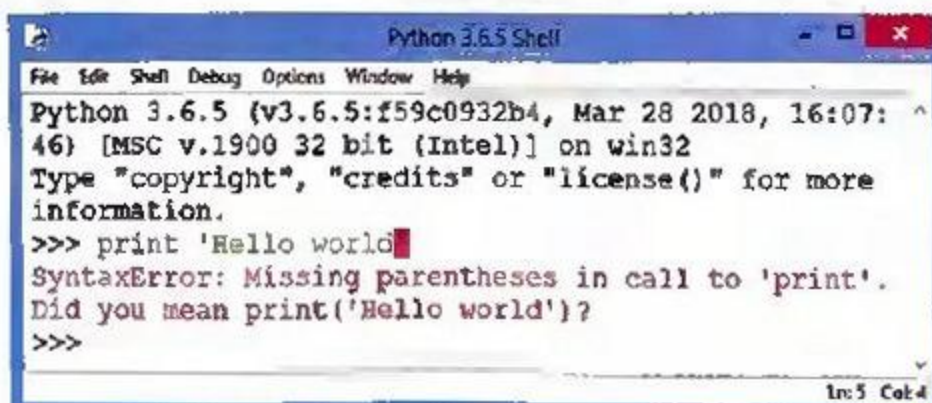


Fig. 5.20: Syntax Error: Invalid Syntax

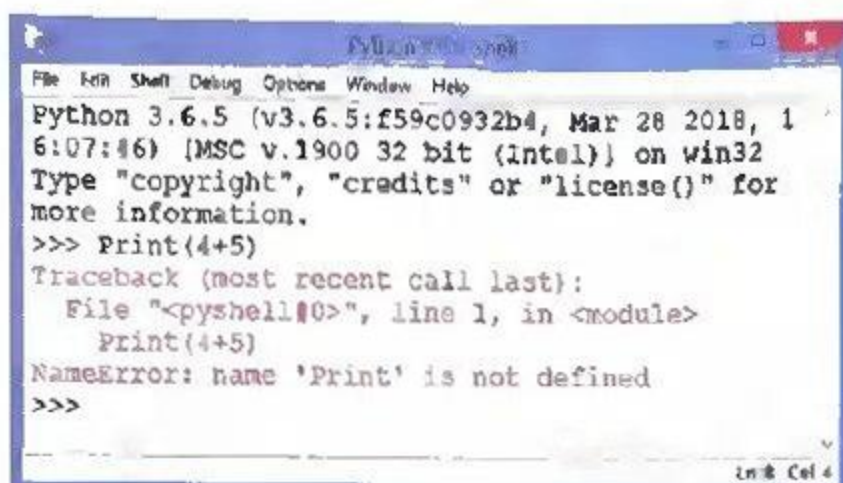
The given statement is an example of syntax error as it violates the language protocol by not giving parentheses() with the print() function. So, the corrected statement should be:

```
>>> print('Hello world')
```

```
Hello world
```

However, some syntactical errors are quite hard to find. Python is case-sensitive, so you may use the wrong case for a variable and find out that the variable isn't quite working as you thought it would.

For example,



```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> Print(4+5)
Traceback (most recent call last):
  File "<pyshell>", line 1, in <module>
    Print(4+5)
NameError: name 'Print' is not defined
>>>
```

In the above example, we have typed Print() statement instead of print(), which is syntactically wrong and needs to be corrected. So, the correct statement shall be:

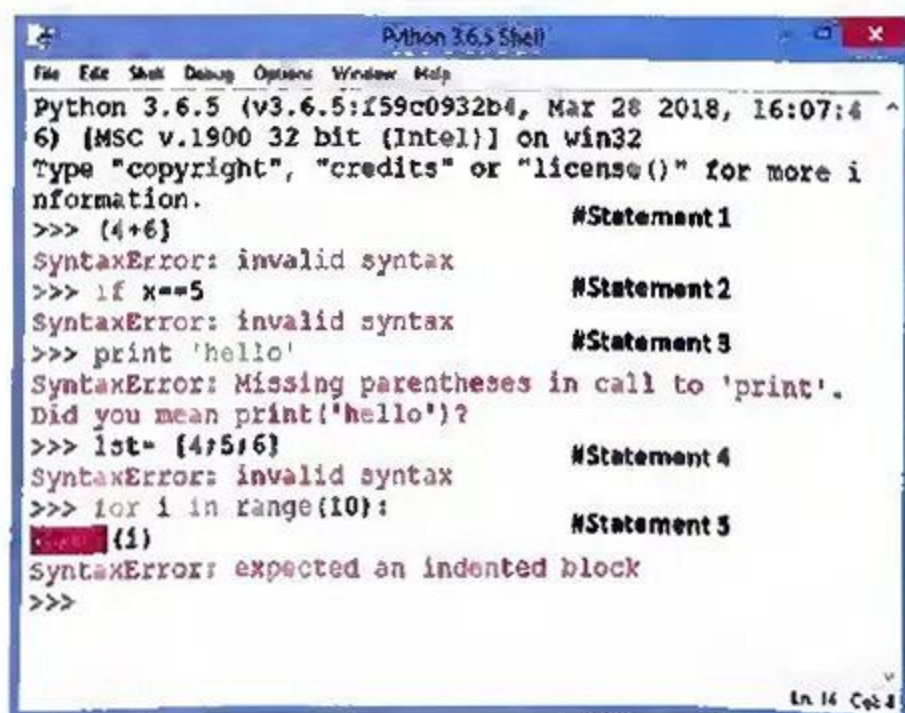
```
>>> print(4+5)
```

```
9
```

Most syntactical errors occur at the time of program execution and the interpreter points them out for you. Fixing the error is made easy because the interpreter generally tells you what to fix and with considerable accuracy.

CTM: Syntax errors are errors that occur due to incorrect format of a Python statement. They occur while the statement is being translated to machine language and before being executed.

A few more examples of syntax errors are shown below in Fig. 5.21.



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> (4+6) #Statement 1
SyntaxError: invalid syntax
>>> if x==5 #Statement 2
SyntaxError: invalid syntax
>>> print 'hello' #Statement 3
SyntaxError: Missing parentheses in call to 'print'.
Did you mean print('hello')?
>>> list= {4,5,6} #Statement 4
SyntaxError: invalid syntax
>>> for i in range(10): #Statement 5
(1)
SyntaxError: expected an indented block
>>>
```

Fig. 5.21: Other types of Syntax Errors

Let us understand the reason behind the occurrence of each error given in the above example.

1. Statement 1 generates syntax error because of improper closing bracket (square bracket instead of parenthesis).
2. Statement 2 generates syntax error because of missing colon (:) after 'if' keyword to end if statement.
3. Statement 3 generates syntax error because of missing parentheses () with print() method.
4. Statement 4 generates syntax error because of use of semicolon instead of comma in a list declaration.
5. Statement 5 generates syntax error because of improper indentation.

5.14.2 Run-time Error

A run-time error occurs after Python interpreter interprets the code you write and the computer begins to execute it. Run-time errors come in different types and some are hard to find.

You know you have a run-time error when the application suddenly stops running and displays an error (exception) dialog box or when the user complains about erroneous output. It usually results in abnormal program termination during execution.

For example,

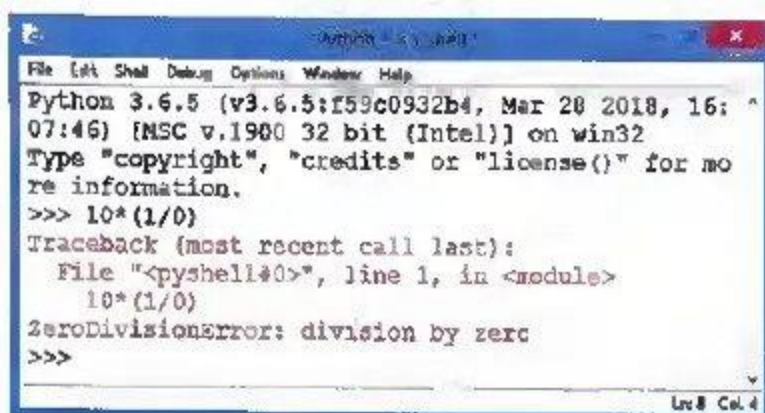


Fig. 5.22: Example of Run-time Error

The above statement is syntactically correct but won't yield any output. Instead, it shall generate an error as division of any number by 0 will result in an error upon execution and illegal program termination. Some examples of Python run-time errors are:

1. division by zero,
2. performing an operation on incompatible types,
3. using an identifier variable which has not been defined,
4. accessing a list element, dictionary value or object attribute which doesn't exist,
5. trying to access a file which doesn't exist.

CTM: Errors are exceptional, unusual and unexpected situations and they are never part of the normal flow of a program.

In Python, such unusual situations are termed as Exceptions. Exceptions are usually run-time errors.

5.14.3 Logical Errors

A logical error/bug (called semantic error) does not stop execution but the program behaves incorrectly and produces undesired/wrong output. Since the program interprets successfully even when logical errors are present in it, it is sometimes difficult to identify these errors.

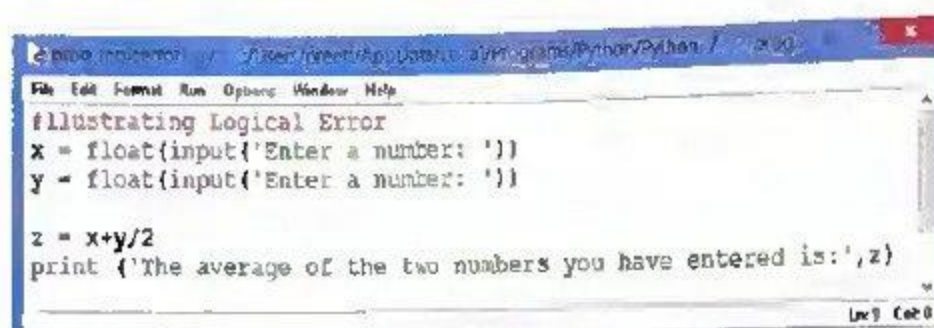
Logical errors are the most difficult to fix. They occur when the program runs without crashing but produces an incorrect result. The error is caused by a mistake in the program's logic. You won't get an error message because no syntax or run-time error has occurred.

Some examples of logical errors are:

- Using the wrong variable name for calculations.
- Using integer division or modulus operator in place of division operator.
- Giving operator Precedence wrong.

You will have to find the problem on your own by reviewing all the relevant parts of the code. For example, if we wish to find the average of two numbers, 10 and 12, and we write the code as $10 + 12/2$, it would run successfully and produce the result 16, which is wrong. The correct code to find the average should have been $(10 + 12)/2$ to get the output as 11.

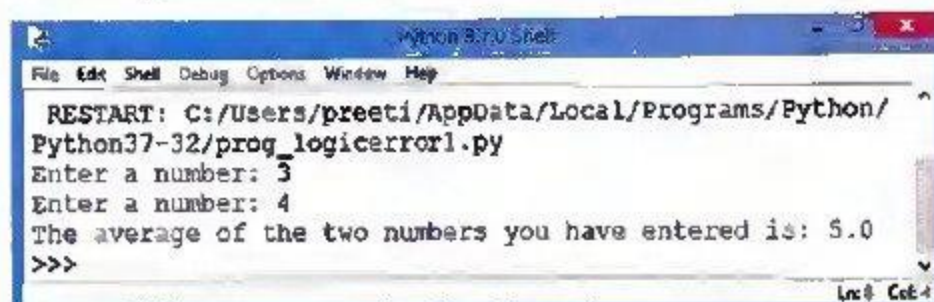
For example,



```
File Edit Format Run Options Window Help
#Illustrating Logical Error
x = float(input('Enter a number: '))
y = float(input('Enter a number: '))

z = x+y/2
print ('The average of the two numbers you have entered is:',z)
```

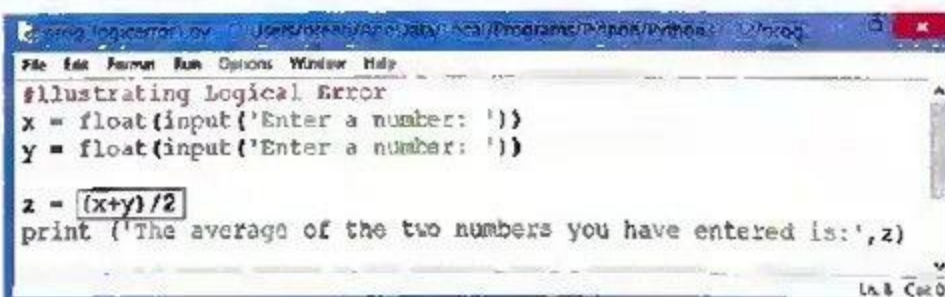
The example above should calculate the average of the two numbers the user enters. But because of the order of operations in arithmetic (division is evaluated before addition), the program will not give the right answer; this is known as logical error.



```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_logicerror1.py
Enter a number: 3
Enter a number: 4
The average of the two numbers you have entered is: 5.0
>>>
```

As a result, you will get 5.0 as the output instead of 3.5.

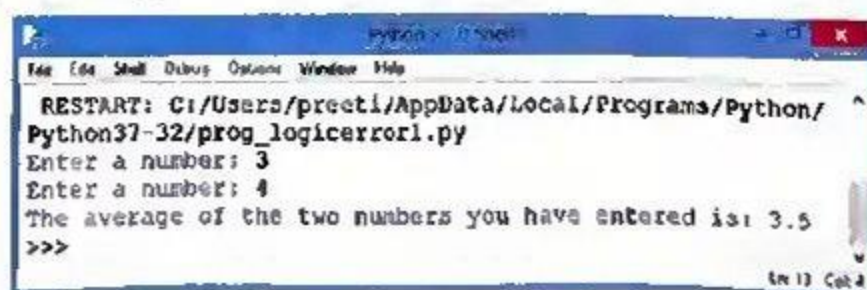
To rectify this problem, we will simply add parentheses: $z = (x+y)/2$



```
File Edit Format Run Options Window Help
#Illustrating Logical Error
x = float(input('Enter a number: '))
y = float(input('Enter a number: '))

z = (x+y)/2
print ('The average of the two numbers you have entered is:',z)
```

Hence, now we will get the right result as 3.5.



```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_logicerror1.py
Enter a number: 3
Enter a number: 4
The average of the two numbers you have entered is: 3.5
>>>
```



MEMORY BYTES

- A Python program can contain various components like expressions, statements, comments, functions, blocks and indentation.
- An expression is a legal combination of symbols that represents a value
- A statement is a programming instruction.
- Comments are non-executable, additional information added in a program for readability
- In Python, comments begin with the hash (#) symbol/character
- A variable in Python is defined only when some value is assigned to it.
- Python supports dynamic typing, i.e., a variable can hold values of different types at different times
- A function is a named block of statements that can be invoked by its name
- The input() function evaluates the data input and takes the result as numeric type.
- Output is generated through print statement.
- An expression is a valid combination of operators and operands. Operands are the objects on which operators are applied.
- Arithmetic, relational and logical operators are used for performing computations in a program.
- A string is a sequence of characters. To specify a string, we may use single, double or triple quotes
- The operator + (addition) applied on two string operands is used for concatenating them. Similarly, the operator * (multiplication) is used to repeat a string a specific number of times.
- When relational operators are applied in strings, strings are compared left to right, character by character, according to their ASCII values.
- While evaluating a Boolean expression involving 'and' operator, the second sub-expression is evaluated only if the first sub-expression yields True.
- While evaluating an expression involving 'or' operator, the second sub-expression is evaluated only if the first sub-expression yields False.
- A variable is a name that refers to a value. We may also say that a variable associates a name with a data object such as number, character, string or Boolean.
- Values are assigned to variables using assignment statement. The syntax for assignment statements is as follows
`variable = expression`
 where variable is called l-value and expression is called r-value.
- In an assignment statement, the expression on the right-hand side of the equal to operator (=) is evaluated and the value so obtained is assigned to the variable on the left-hand side of the = operator.
- A variable name must begin with a letter or _ (underscore character). It may contain any number of letters, digits or underscore characters. No other character apart from these is allowed.
- Python is case-sensitive. Thus, marks and Marks are different variables.
- The shorthand notion for `a = a <operator> b` is
`a <operator>= b`
- In Python, multiple assignment statements can be specified in a single line as follows:
`<name1>, <name2>, ... = <expression1>, <expression2>, ...`
- A keyword is a reserved word that is already defined by Python for a specific use. Keywords cannot be used for any other purpose.
- Trying to use a variable that has not been assigned a value gives an error.
- Run-time errors occur during the execution of a program.
- Irregular unexpected situations occurring during run-time are called exceptions.

OBJECTIVE TYPE QUESTIONS

1. Fill in the blanks.

- (a) The smallest individual unit in a program is known as a
- (b) A word having special meaning reserved by a programming language is known as
- (c) A literal is a sequence of characters surrounded by quotes.
- (d) are tokens that trigger same computation/action when applied to variables.
- (e) A is a reserved named location that stores values.
- (f) To determine the type of an object we use function.
- (g) The input() always returns a value of type.
- (h) Blocks of codes are represented through
- (i) In typing, a variable can hold values of different types at different times.
- (j) In Python, the floating point numbers have precision of digits.
- (k) Operators that act upon two operands are referred to as operators.
- (l) truncates fractional remainders and gives only the whole part as the result.
- (m) A error does not stop execution but the program behaves incorrectly and produces undesired/wrong output.
- (n) Trying to access a variable or file that doesn't exist is a kind of error.

2. State whether the following statements are True or False.

- (a) Python supports Unicode coding standard.
- (b) An identifier must be a keyword of Python.
- (c) Default variable initialization is string literal.
- (d) $3+C=A$ is a valid statement.
- (e) The input() always returns a value of an integer type.
- (f) The print() without any value or name or expression prints a blank line.
- (g) In Python, Boolean type is a sub-type of Integer.
- (h) "AISSCE-2020" is a valid string in Python.
- (i) In Python, Integer data type has a fractional part.
- (j) Null literal in Python means "there's nothing here".
- (k) Python supports multiple assignment to multiple variables.
- (l) id() function is used to determine the data type of a variable.

3. Multiple Choice Questions (MCQs):

- (a) Which of the following are the fundamental building blocks of a Python program?
(i) Identifiers (ii) Constants (iii) Punctuators (iv) Tokens
- (b) Identifier name cannot be composed of special characters other than
(i) # (ii) Hyphen (-) (iii) \$ (iv) Underscore (_)
- (c) Python takes indented spaces after the function declaration statement by default.
(i) 5 (ii) 6 (iii) 4 (iv) 3
- (d) Single line comments in Python begin with symbol.
(i) # (ii) " (iii) ''' (iv) %
- (e) Which of the following does not represent a complex number?
(i) $K = 2 + 3j$ (ii) $K = \text{complex}(2,3)$ (iii) $K = 2 + 3i$ (iv) $K = 4 + 3j$
- (f) What is the order of precedence of arithmetic operators given below in Python?
1. Parenthesis 2. Exponential 3. Multiplication
4. Division 5. Addition 6. Subtraction
(i) 1,2,3,4,5,6 (ii) 2,3,4,5,6,1 (iii) 1,3,2,6,4,5 (iv) 4,6,5,2,3,1

- (g) What will be the output of the following snippet?
- ```
x, y = 2, 6
x, y = y, x + 2
print(x, y)
```
- (i) 6 6                      (ii) 4 4                      (iii) 4 6                      (iv) 6 4
- (h) The extension of a Python file is given as—
- (i) .pt                      (ii) .pwy                      (iii) .py or .pyw                      (iv) .yppy
- (i) The reserved words used by Python Interpreter to recognize the structure of a program are termed as .....
- (i) Identifiers                      (ii) Tokens                      (iii) Literals                      (iv) Keywords
- (j) What can be the maximum possible length of an identifier?
- (i) 31                      (ii) 63  
(iii) 79                      (iv) Can be of any length
- (k) Which of the following operators is floor division?
- (i) +                      (ii) /                      (iii) //                      (iv) >
- (l) Which of the following is an invalid statement?
- (i) a=b=c=2                      (ii) a,b,c=10,20,30  
(iii) a b c = 20,30,40                      (iv) a\_b\_c=20

## SOLVED QUESTIONS

1. What are the different components of a variable?

**Ans.** A variable has three main components:

- Identity (memory address) of the variable.
  - Type (data type) of the variable.
  - Value of the variable.
2. Consider the following statements in Python interpreter and describe the output/statement required:
- Print a message "Hello World".
  - ```
a = 10
b = 12
c = a + b
print(c)
```
 - To retrieve the data type of the inputted string "Hello" stored inside the variable 'a'.
 - To describe the data type of variable 'b'.
 - To retrieve the address of variables a and b.
 - State the output:


```
>>> b = 12
>>> d = b
>>> d
>>> b
>>> id(d)
>>> id(b)
```
 - ```
>>> a = "Hello"
>>> a * 10
```
  - To display the value for  $a^2$ ,  $a^3$  and  $a^4$
  - ```
>>> a = 15
>>> b = 4
>>> a/b
>>> a//b
```

(j) To swap the values of two variables, a and b, using multiple assignment statements.

```
(k) >>> a,b,c,d=10,27,4+5,10-7
>>> a//d
>>> d**d
>>> b/c
>>> a+b+c-d
```

Ans. (a) >>> print("Hello World")
Hello World

```
(b) >>> a = 10
>>> b = 12
>>> c = a + b
>>> print(c)
22
```

```
(c) >>> a = "Hello"
>>> type(a)
<class 'str'>
```

```
(d) >>> type(b)
<class 'int'>
```

```
(e) >>> id(a)
52193248
>>> id(b)
1616896960
```

```
(f) >>> d = b
>>> d
12
>>> b
12
>>> id(d)
1616896960
>>> id(b)
1616896960
```

```
(g) >>> a = "Hello"
>>> a*10
'HelloHelloHelloHelloHelloHelloHelloHelloHello'
```

```
(h) >>> a = 10
>>> a**2
100
>>> a**3
1000
>>> a**4
10000
```

```
(i) >>> a = 15
>>> b = 4
>>> a/b
3.75
>>> a//b
3
```

```

(j) >>> a, b = 10, 12
    >>> a, b = b, a
    >>> a
    12
    >>> b
    10
(k) >>> a, b, c, d = 10, 27, 4+5, 10-7
    >>> a//d
    3
    >>> d**d
    27
    >>> b/c
    3.0
    >>> a+b+c-d
    43

```

3. What is the difference between a keyword and a variable?

Ans. Keyword is a special word that has a specific purpose and meaning to the Python interpreter. Keywords are reserved words that can be used in our program only for the purpose for which they have been defined.

For example, int, float, for, if, elif, else, etc., are keywords.

Variable is the user-defined name given to a value. Variables are not fixed/reserved. These are defined by the user but they can have letters, digits and the symbol underscore. They must begin with either a letter or underscore. *For example, _age, name, result_1, etc.*

4. What is a function? How is it useful?

Ans. A function in Python is a named block of statements within a program that performs a specific task.

For example, the following program has a function within it, namely greet_msg().

```

# prog1.py
def greet_msg():
    print("Hello there!!")
    name = input("Your name:")
    print(name, greet_msg())

```

Functions are useful as they can be reused anywhere through their function call statements. So, for the same functionality, one need not rewrite the code every time it is needed; rather, through functions it can be used again and again.

5. Find the error.

```

def minus(total, decrement)
    output = total - decrement
    return output

```

Ans. The function header has colon missing at the end. So, we need to add colon (:) at the end of the header line.

Thus, the corrected code will be:

```

def minus(total, decrement):
    output = total - decrement
    return output

```

6. Differentiate between mutable and immutable objects

Ans. *Mutable object:* Objects whose values can be changed after they are created and assigned values are called mutable objects. *For example, list, dictionary, etc.*

Immutable object: Objects whose values cannot be changed after they are created and assigned values are called immutable objects. *For example, int, float, string, tuple, etc.*

7. Ritu is confused between $3*2$ and $3**2$. Help her to know the difference between the two expressions.

Ans. The statement $3*2$ multiplies 3 by 2 and produces the result 6 while $3**2$ calculates 3 raised to the power 2 and produces the result 9.

8. How many types of strings are supported in Python?

Ans. Python allows two string types:

- (1) Single line Strings—Strings that are terminated in a single line enclosed within single and double quotes.
- (2) Multiline Strings—Strings storing multiple lines of text enclosed within three single or double quotes.

9. Differentiate between explicit and implicit type conversion.

Ans. An implicit type conversion is a conversion performed by the compiler without programmer's intervention. An implicit conversion is applied generally whenever multiple data types are intermixed in an expression (mixed mode expression), so as not to lose information.

An explicit type conversion is user-defined conversion that forces an expression to a specific type. The explicit type conversion is also known as Type Casting.

10. What are variable-naming conventions?

- Ans. (i) A variable must start with a letter or underscore followed by any number of digits and/or letters.
(ii) No reserved word or standard should be used as the variable name.
(iii) No special character (other than underscore) should be included in the variable name.
(iv) Case sensitivity should be taken care of.

11. What is None in Python?

Ans. Python has one special data type called None. The None type is used to indicate something that has not yet been created. It is a non-null, unidentified value.

12. Identify the types of the following literals:

23.789, 23789, True, {4: "four", 5: "five"}, 'True', "True", (1,2,3), False, "False", None, [100,200,300]

Ans. 23.789	Floating point
23789	integer
True	Boolean
{4: "four", 5: "five"}	Mapping (Dictionary)
'True'	String
"True"	String
(1,2,3)	Sequence (Tuple)
False	Boolean
"False"	String
None	None
[100,200,300]	Sequence (List)

13. What is the difference between 55L and 55?

Ans. An l or L suffix indicates that it is a long integer constant. Thus, 55L is a long integer value and 55 is an integer value.

14. Write the corresponding Python statements:

- (i) Assign value 100 to variable A and value 200 to variable B.
- (ii) Assign value 10 to variables a, b, and c.
- (iii) Assign value 20 to x and delete 5 from x and assign the variable x to variable y.

Ans. (i) A=100
B=200
(ii) a=b=c=10
(iii) x=20
x=x-5
y=x

15. Find the output generated by the following code:

(1) <code>x=2</code> <code>y=3</code> <code>x+=y</code> <code>print(x,y)</code> (Ans. 5 3)	(2) <code>x=8</code> <code>y=2</code> <code>x+=y</code> <code>y-=x</code> <code>print(x,y)</code> (Ans. 10 -8)
(3) <code>a=5</code> <code>b=10</code> <code>a+=a+b</code> <code>b*=a+b</code> <code>print(a,b)</code> (Ans. 20 300)	(4) <code>p=10</code> <code>q=20</code> <code>p*=q//3</code> <code>q+=p+q**2</code> <code>print(p,q)</code> (Ans. 60 480)
(5) <code>p=5/2</code> <code>q=p*4</code> <code>r=p+q</code> <code>p+=p+q+r</code> <code>r+=p+q+r</code> <code>q-=p+q*r</code> <code>print(p,q,r)</code> (Ans. 27.5 -642.5 62.5)	(6) <code>p=2/5</code> <code>q=p*4</code> <code>r=p*q</code> <code>p+=p+q-r</code> <code>r*=p-q+r</code> <code>q+=p+q</code> <code>print(p,q,r)</code> (Ans. 1.7599999999999998 4.96 0.512)

16. What is the difference between an expression and a statement in Python?

Ans.

S.No.	Expression	Statement
1.	An expression is a combination of symbols, operators and operands.	Statement is defined as any programming instruction given in Python as per the syntax.
2.	An expression represents some value.	Statement is given to execute a task.
3.	The outcome of an expression is always a value.	Statement may or may not return a value as the result.
4.	For example, <code>3 * 7 + 10</code> is an expression.	<code>print("Hello")</code> is a statement.

17. Identify the error in the following Python statement:

```
>>> print("My name is", first_name)
```

Write the corrected statement also.

Ans. The above statement is trying to print the value of an undefined variable `first_name`. The correction is made by defining the variable name before using it, i.e.,

```
>>> first_name = 'Rinku'
```

```
>>> print("My name is", first_name)
```

18. The following code is not giving the desired output. We want to input the value as 15 and obtain output as 30. Could you pinpoint the problem?

```
num = input("Enter Number:")
```

```
result = num * 2
```

```
Print(result)
```

Ans. The problem is with `input()` function as it returns value as a string, so when we the input the value 15 it will assign to `num` variable as string '15' and not as integer 15. So, when `num` variable is multiplied by 2, the output is '1515' and not 30. To solve this problem, `int()` function is to be used while accepting the input. Also, `Print` is not a legal statement of Python; it should be `print`. The corrected code is:

```
num = int(input("Enter Number:"))
```

```
result = num * 2
```

```
print(result)
```

19. Write a program to obtain temperature in Celsius and convert it into Fahrenheit using the formula:
 $F = C \times 9/5 + 32$

Ans. #Celsius to Fahrenheit

```
cels = float(input("Enter Temp in Celsius :"))  
print("Temperature in Celsius is :", cels)  
f = cels * 9/5 + 32  
print("Temperature in Fahrenheit is :", f)
```

20. What will be the output produced by the following code?

```
name = 'Neeru'  
age = 21  
print(name, "you are", 21, "now but",)  
print("you will be", age + 1, "next year")
```

Ans. Output should be:

Neeru, you are 21 now but
you will be 22 next year

21. What will be the output of the following code? Explain.

```
x, y = 2, 6  
x, y = y, x + 2  
print(x, y)
```

Ans. 6 4

Explanation: The first line of code assigns values 2 and 6 to variables x and y respectively.

The next line (second line) of code first evaluates the right-hand side, i.e., $y, x + 2$, which is 6, 2 + 2, i.e., 6, 4;

And then assigns this to x, y, i.e., $x, y = 6, 4$; so x gets 6 and y gets 4.

The third line prints x and y so the output is 6 4.

22. Write a Python code to calculate and display the selling price of an item. Cost price and profit is to be accepted by user.

```
Ans. costPrice = int(input('Enter cost price:'))  
profit = int(input('Enter profit:'))  
sellingPrice = costPrice + profit  
print('Selling Price:', sellingPrice)
```

Output:

Enter cost price: 50
Enter profit: 12
Selling Price: 62

23. What output will be produced by the following code:

```
A, B, C, D = 9.2, 2.0, 4, 21  
print(A/4)  
print(A//4)  
print(B**C)  
print(A%C)
```

Ans. 2.3

2.0

16.0

1.1999999999999999

24. Write the Python code to convert the time given in minutes into hours and minutes.

```
Ans. min=int(input("Enter time in minutes"))  
h=min//60  
m=min%60  
print("Hours=",h)  
print("Minutes=",m)
```

25. Evaluate (to true or false) each of the following expressions:

- (i) $14 < -14$
- (ii) $14 < 14$
- (iii) $-14 > 14$
- (iv) $15 >= 15$

Ans. (i) True
(ii) False
(iii) True
(iv) False

26. A year is a leap year if it is divisible by 4, except that years divisible by 100 are not leap years unless they are also divisible by 400. Write a program that asks the user for a year and print whether it is a leap year or not.

Ans.

```
year = int(input("Please Enter the Year Number you want: "))
if year%400 == 0 or year%4 == 0 and year%100 != 0:
    print(year, "is a Leap Year")
else:
    print(year, "is Not a Leap Year")
```

27. Write a Python program that accepts cost price and quantity of pencil from the user. Calculate and display total price.

Ans.

```
costprice = int(input('Enter cost price of pencil: '))
qty = int(input('Enter quantity: '))
totalprice = costprice * qty
print("Total price is:", totalprice)
```

28. Differentiate between Syntax and Run-time errors. Give an appropriate example.

Ans. Syntax Error: Violation of formal rules of a programming language such as misspelling a keyword or incorrect indentation, etc., results in syntax error.

For example,

```
len('hello') = 5
File "<stdin>", line 1
SyntaxError: can't assign to function call
```

Run-time Error: These errors are generated during a program execution due to resource limitation such as division by zero, trying to access an identifier or file which doesn't exist, etc. Python has provision of check points to handle these errors known as Exception Handling.

For example,

```
a=10
b=0
c=a/b
would result in
ZeroDivisionError: division by zero
```

29. Explain the concept of R-value & L-value with example.

Ans. In a normal expression, the variable to the left side of an assignment operator refers to L-value (left-value) because it resides in memory and is addressable, and an expression after the assignment operator refers to R-value (right-value). It does not represent an object occupying some identifiable location in memory.

For example, $X=1$ #X is an L-value

$Y=20$ # Y is an L-value

$Z=X+Y$ # X+Y is an R-value

30. Give the outputs for the following expressions:

- (i) $(5 < 10)$ and $(10 < 5)$ or $(3 < 18)$ and not $8 < 18$

```

(ii) P, Q, R = 1, 2, 3
X = P + Q + R
Y = P + Q + R * 10
Z = P + (Q + R) * 10
print("X=", X)
print("Y=", Y)
print("Z=", Z)

```

Ans. (i) 1234
 (ii) X=1000
 Y=1234
 Z=4567

UNSOLVED QUESTIONS

- Write Python command/instruction/statement to display your name
- Write Python command to display your school name, class and section, separated by "."
- Evaluate the following expressions manually:
 - $(2 + 3) ** 3 - 6 / 2$
 - $(2 + 3) * 5 // 4 + (4 + 6) // 2$
 - $12 + (3 * 4 - 6) / 3$
 - $12 + (3 * 4 - 6) // 2$
 - $12 * 3 \% 5 + 2 * 6 // 4$
 - $12 \% 5 * 3 + (2 * 6) // 4$
- Evaluate the above expressions by using IDLE as a calculator and verify the results that you get manually.
- Identify invalid variable names from the following, giving reason for each:
 Group, if, int, total marks, S.I., volume, tot_strength, #tag, tag\$, 9a
- Find the output of the following code:

```

(1) x=3
    y=x+2
    x+=y
    print(x,y)

```

```

(2) x=-2
    y=2
    x+=y
    y-=x
    print(x,y)

```

```

(3) a=5
    b=2*a
    a+=a+b
    b*=a+b
    print(a,b)

```

```

(4) p=10
    q=20
    p*=q/3
    q+=p+q*2
    print(p,q)

```

```

(5) p=5%2
    q=p**4
    r=p//q
    p+=p+q+r
    r+=p+q+r
    q-=p+q*r
    print(p,q,r)

```

```

(6) p=21//5
    q=p%4
    r=p*q
    p+=p+q-r
    r*=p-q+r
    q+=-p+q
    print(p,q,r)

```

- Write Python expressions equivalent to the following arithmetic/algebraic expressions:

(1) $\frac{a+b}{2}$

(2) $\frac{3^2 + 9^3}{2}$

(3) $3^2 + \frac{9^3}{5}$

(4) $\sqrt{a} + \frac{a+2}{b}$

(5) $8 - 6 + \frac{6 * sum}{7} - \sqrt{var}$

(6) $ut + \frac{1}{2}at^2$

8. Write Python expressions to represent the following situations:
 - (a) Add remainder of 8/3 to the product of 8 and 3.
 - (b) Find the square root of the sum of 8 and 43.
 - (c) Find the sum of the square roots of 8 and 43.
 - (d) Find the integral part of the quotient when 100 is divided by 32.
9. What are operators? Give some examples of unary and binary operators.
10. What is an expression and a statement?
11. What all components can a Python program contain?
12. What are variables? How are they important for a program?
13. What is Dynamic Typing feature in Python?
14. Write a function called `calculate_area()` that takes base and height as an input argument and returns an area of a triangle as an output. The formula used is:

$$\text{Area of a Triangle} = \frac{1}{2} * \text{base} * \text{height}$$
15. Modify the above function to take a third parameter called shape type. Shape type should be either triangle or rectangle. Based on the shape, it should calculate the area. Formula used:

$$\text{Area of a Rectangle} = \text{length} * \text{width}.$$
16. Write a function called `print_pattern` that takes integer number as argument and prints the following pattern if the input number is 3:


```

*
**
***
      
```

 If input is 4, then it should print:


```

*
**
***
****
      
```
17. Write a function that takes amount-in-dollars and dollar-to-rupee conversion price; it then returns the amount converted to rupees.
18. What would the following code do?


```

a = b = 70
      
```
19. What is the error in the following code?


```

z, p = 6
      
```
20. Find out the error(s) in the following code fragments?
 - (1) `temperature = 90`
`Print temperature`
 - (2) `a = 30`
`b = a + b`
`print(a And b)`
 - (3) `a, b, c = 2, 8, 9`
`print(a, b, c)`
`c, b, a + a, b, c`
`print(a ; b ; c)`
 - (4) `x = 24`
`4 = x`
 - (5) `Print("X =" X)`

21. What will be the output produced by the following code fragment(s)?

```
(1) X = 10
    X = X + 10
    X = X - 5
    print(X)
    X, Y = X - 2, 22
    print(X, Y)
```

```
(2) first = 2
    second = 3
    third = first + second
    print(first, second, third)
    third = second * first
    print(first, second, third)
```

```
(3) side = int(input('side'))
    #side given as 7
    area = side * side
    print(side, area)
```

22. "Comments are useful and an easy way to enhance readability and understandability of a program." Elaborate with examples.

23. Find errors in the following code fragments.

```
(1) a = 3
    print(a)
    b = 4
    print(b)
    s = a+b
    print(s)
```

```
(2) Name = "Prateek"
    Age = 26
    Print("your name & age are", Name + Age)
```

```
(3) A = 3
    S = A + 10
    A = "New"
    Q = A / 10
```

24. Give the output.

```
x = 40
y = x + 1
x = 20, y + x
print(x, y)
```

25. Give the output.

```
x, y = 20, 60
y, x, y = x, y - 10, x + 10
print(x, y)
```

26. Give the output.

```
a, b = 12, 13
c, b = a*2, a/2
print(a, b, c)
```

27. Give the output.

```
a, b, c = 10, 20, 30
p, q, r = c - 5, a + 3, b - 4
print('a and b, : ', p, q, r)
```

28. Find errors in the following code fragment. (The input entered as XI)

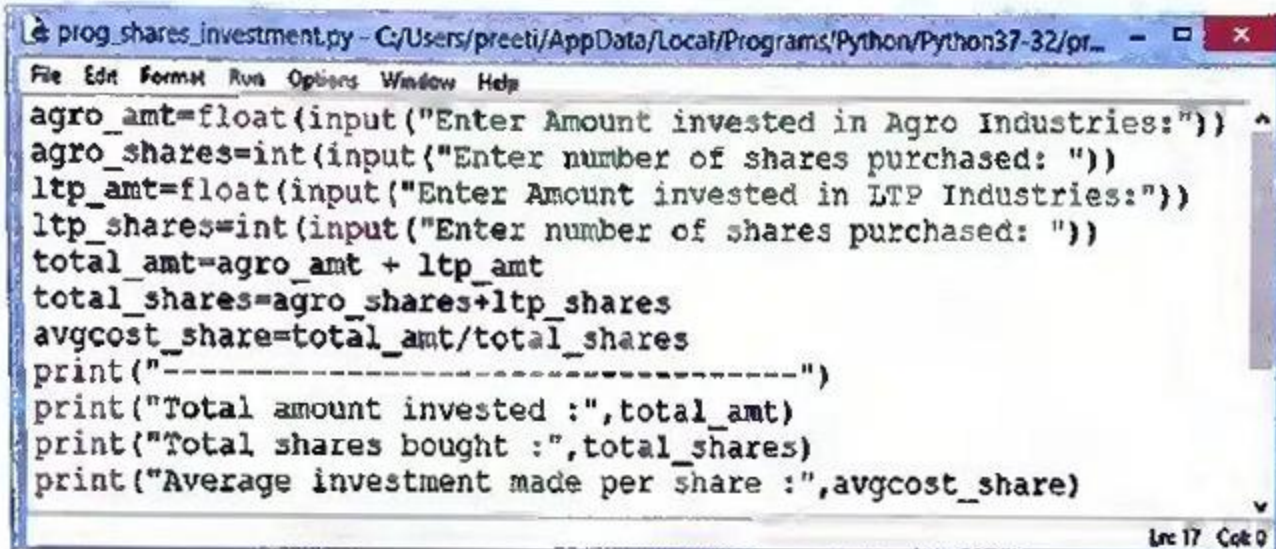
```
c = int(input("Enter your class:"))
print("Your class is", c)
```
29. Find errors in the following code fragment.

```
cl= input("Enter your class")
print("Last year you were in class" cl-1)
```
30. Write a Python program that accepts radius of a circle and prints its area.
31. Write Python program that accepts marks in 5 subjects and outputs average marks.
32. Write a program to find the area of a triangle.
33. Write a program to input a number and print its first five multiples.
34. Write a program to read details like name, class, age of a student and then print the details, firstly in the same line and then in separate lines.
35. Write a program to read three numbers in three variables and swap first two variables with the sums of first and second, second and third numbers, respectively.

CASE-BASED/SOURCE-BASED INTEGRATED QUESTIONS

1. Mr. Upendra Awasthi is in share trading business. He has invested ₹ 545000 in 500 shares of Agro Industries Ltd. and ₹ 357550 in 300 shares of LTP Associates (P) Ltd. Write a program to calculate his average investment cost per share.

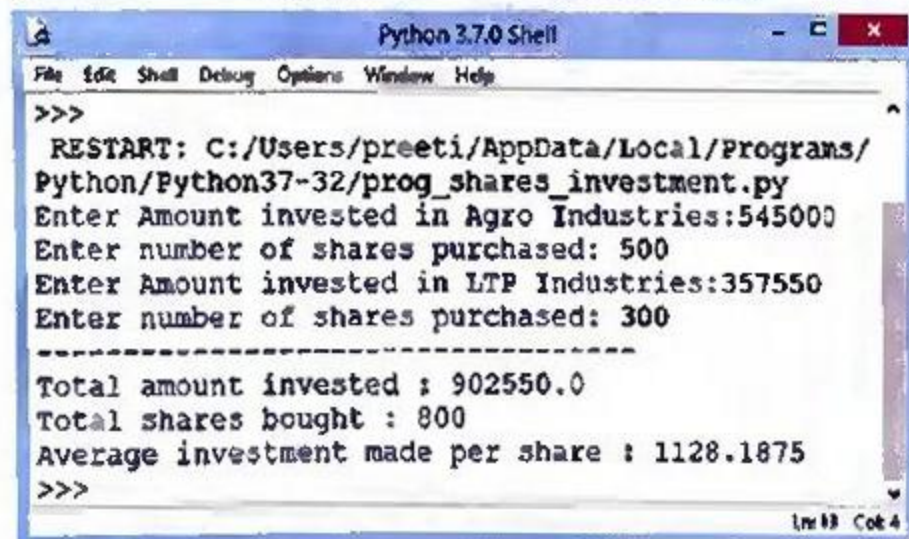
Ans.



```

prog_shares_investment.py - C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/pr...
File Edit Format Run Options Window Help
agro_amt=float(input("Enter Amount invested in Agro Industries:"))
agro_shares=int(input("Enter number of shares purchased: "))
ltp_amt=float(input("Enter Amount invested in LTP Industries:"))
ltp_shares=int(input("Enter number of shares purchased: "))
total_amt=agro_amt + ltp_amt
total_shares=agro_shares+ltp_shares
avgcost_share=total_amt/total_shares
print("-----")
print("Total amount invested :",total_amt)
print("Total shares bought :",total_shares)
print("Average investment made per share :",avgcost_share)

```



```

Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_shares_investment.py
Enter Amount invested in Agro Industries:545000
Enter number of shares purchased: 500
Enter Amount invested in LTP Industries:357550
Enter number of shares purchased: 300
-----
Total amount invested : 902550.0
Total shares bought : 800
Average investment made per share : 1128.1875
>>>

```

2. EX500 is an India-based Cargo company that deals in the export and selling of various goods within the country as well as abroad. To do business analysis of day-to-day transactions, they have hired a programmer to develop a computerized system for the purpose.

Write a Python code that accepts cost of goods sold (cogs), revenue generated (rg), operating costs (oc) and prints gross profit (gp), net profit (np) and net profit percentage (npp). [Net profit = Revenue – cogs – oc]

Ans.

```
File Edit Format Run Options Window Help
#Program to calculate Gross profit, net profit and
#net profit percentage
cogs = int(input('Enter cost of goods sold : '))
rg = int(input('Enter revenue generated : '))
oc = int(input('Enter operating cost : '))
gp = rg - cogs # Gross profit
np = gp - oc # Net profit
npp = np/rg*100 # Net profit percentage
print('Gross profit is', gp)
print('Net profit is', np)
print('Net profit percentage is', npp)
```

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
more information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_revenue1.py
Enter cost of goods sold : 100000
Enter revenue generated : 150000
Enter operating cost : 12000
Gross profit is 50000
Net profit is 38000
Net profit percentage is 25.333333333333336
>>>
```

3. Gurukul Vidyapeeth is a reputed institution in the field of academics and extracurricular activities and involved in the overall growth of every child. Every year, with the commencement of the new session, it hikes fee by 10% for all the students which was done manually till now and required enormous efforts on the part of office staff.

To solve this problem, develop a Python program that automatically calculates this 10% fee hike every year after obtaining the basic fee amount from the user and displays it for the parents of students enrolled with the institute.

Ans.

```
prog_feehike.py - C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_feehike.py
File Edit Format Run Options Window Help
#Program to calculate 10% fee hike
amount = int(input('Enter Amount'))
fee_hike = 0.1*amount
total_fee = amount + fee_hike
print("Fee to pay now:", total_fee)
```

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_feehike.py
Enter Amount 120000
Fee to pay now: 132000.0
>>>
```